

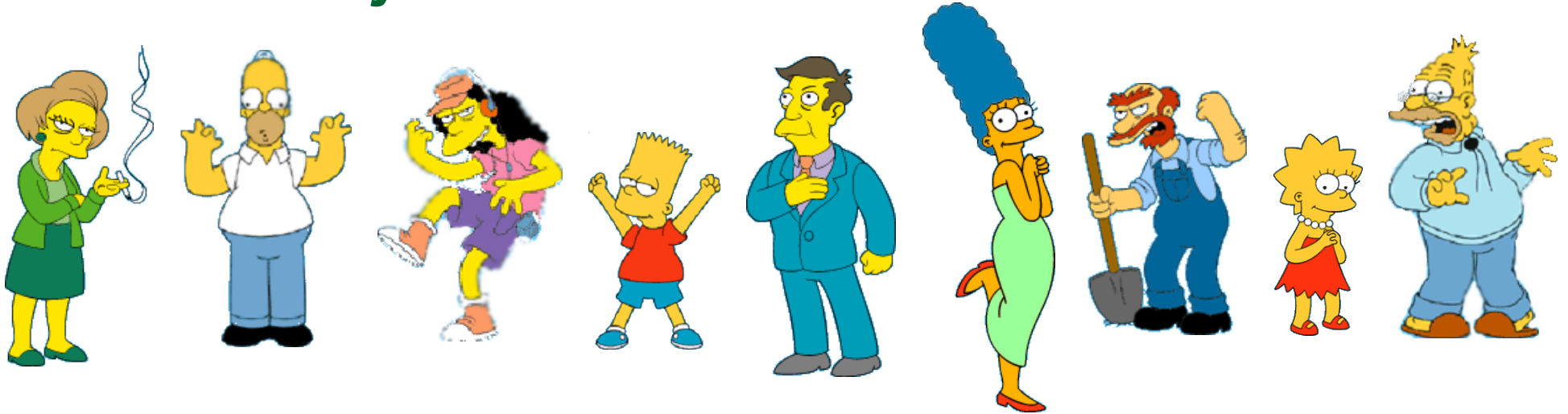
Clustering and Trees

Chapter 10

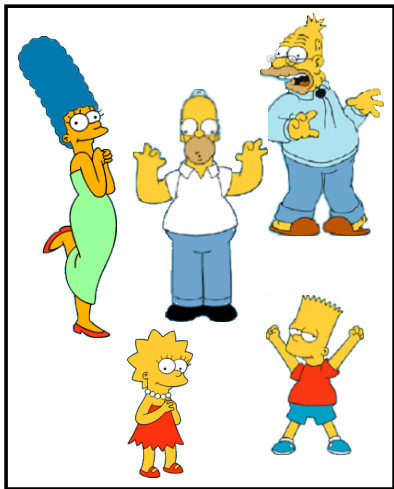
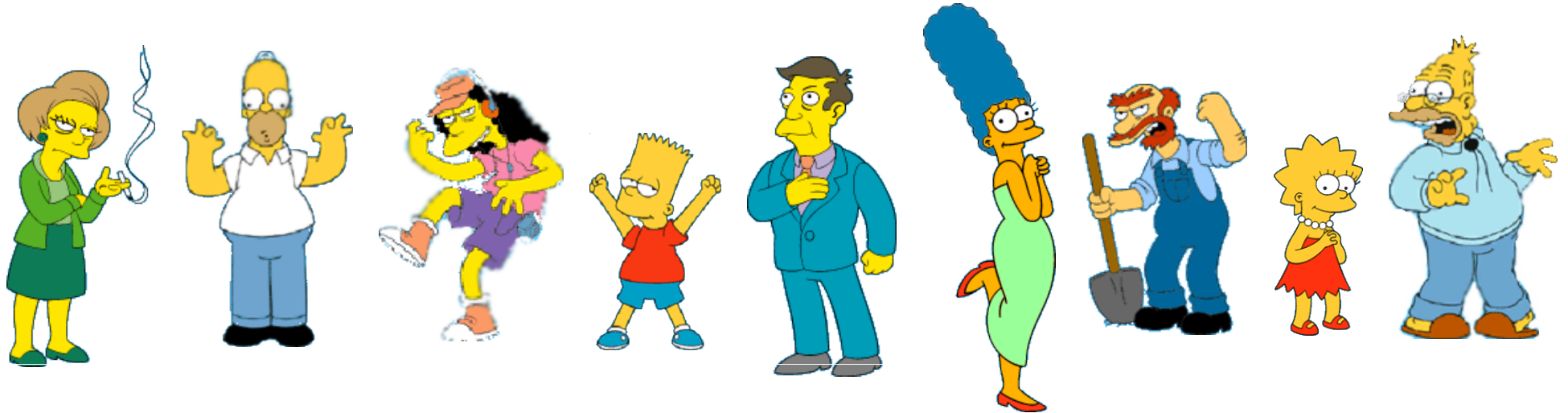
What is clustering?

- Also called ***unsupervised learning***, sometimes called ***classification*** by statisticians, ***sorting*** by psychologists and ***segmentation*** by people in marketing.
- Organizing data into classes such that there is
 - High intra-class similarity
 - Low inter-class similarity
- Finding the class labels and the number of classes directly from the data (in contrast to classification)
- More informally, finding natural groupings among objects.

What is a natural grouping among these objects?



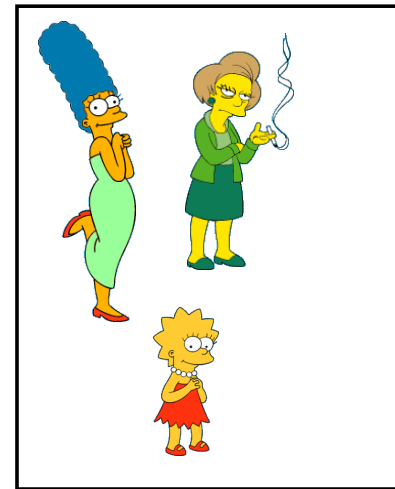
Clustering is subjective



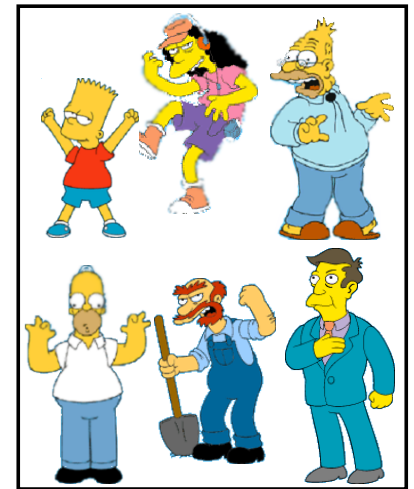
Simpson's Family



School Employees



Females



Males

Clustering Algorithms: Why?

- We have vast amounts of biological data
- When data are viewed as a whole set this data can be perplexing
- When it is clustered into similarity groups it is easier to interpret the data

Inferring Gene Functionality

- Researchers want to know the functions of new genes
- Simply comparing the new gene sequences to known DNA sequences often does not give away the actual function of gene
- For 40% of sequenced genes, functionality cannot be ascertained by only comparing to sequences of other known genes
- Microarrays allow biologists to infer gene function even when there is not enough evidence to infer function based on similarity alone

Clustering of Microarray Data

- Plot each datum as a point in N-dimensional space
- Make a distance matrix for the distance between every two gene points in the N-dimensional space
- Genes with a small distance share the same expression characteristics and might be functionally related or similar!
- Clustering reveal groups of functionally related genes

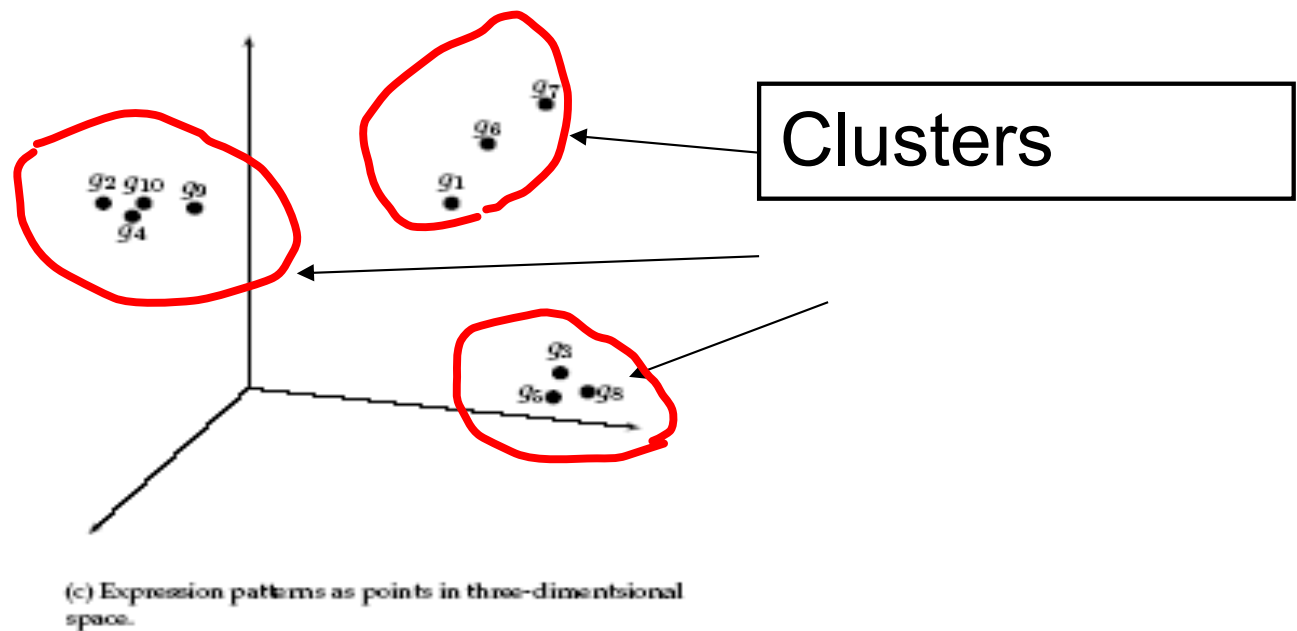
Clustering of Microarray Data (cont' d)

Time	1 hr	2 hr	3 hr
g_1	10.0	8.0	10.0
g_2	10.0	0.0	9.0
g_3	4.0	8.5	3.0
g_4	9.5	0.5	8.5
g_5	4.5	8.5	2.5
g_6	10.5	9.0	12.0
g_7	5.0	8.5	11.0
g_8	2.7	8.7	2.0
g_9	9.7	2.0	9.0
g_{10}	10.2	1.0	9.2

(a) Intensity matrix, I

	g_1	g_2	g_3	g_4	g_5	g_6	g_7	g_8	g_9	g_{10}
g_1	0.0	8.1	9.2	7.7	9.3	2.3	5.1	10.2	6.1	7.0
g_2	8.1	0.0	12.0	0.9	12.0	9.5	10.1	12.8	2.0	1.0
g_3	9.2	12.0	0.0	11.2	0.7	11.1	8.1	1.1	10.5	11.5
g_4	7.7	0.9	11.2	0.0	11.2	9.2	9.5	12.0	1.6	1.1
g_5	9.3	12.0	0.7	11.2	0.0	11.2	8.5	1.0	10.6	11.6
g_6	2.3	9.5	11.1	9.2	11.2	0.0	5.6	12.1	7.7	8.5
g_7	5.1	10.1	8.1	9.5	8.5	5.6	0.0	9.1	8.3	9.3
g_8	10.2	12.8	1.1	12.0	1.0	12.1	9.1	0.0	11.4	12.4
g_9	6.1	2.0	10.5	1.6	10.6	7.7	8.3	11.4	0.0	1.1
g_{10}	7.0	1.0	11.5	1.1	11.6	8.5	9.3	12.4	1.1	0.0

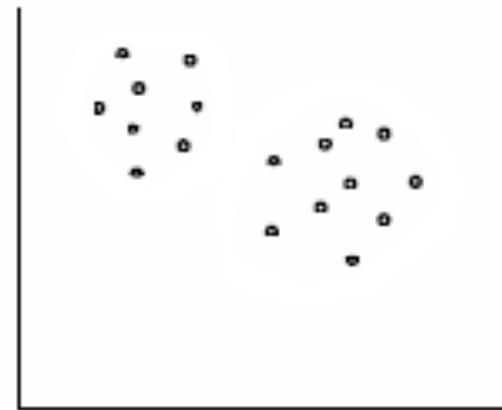
(b) Distance matrix, d



Homogeneity and Separation Principles

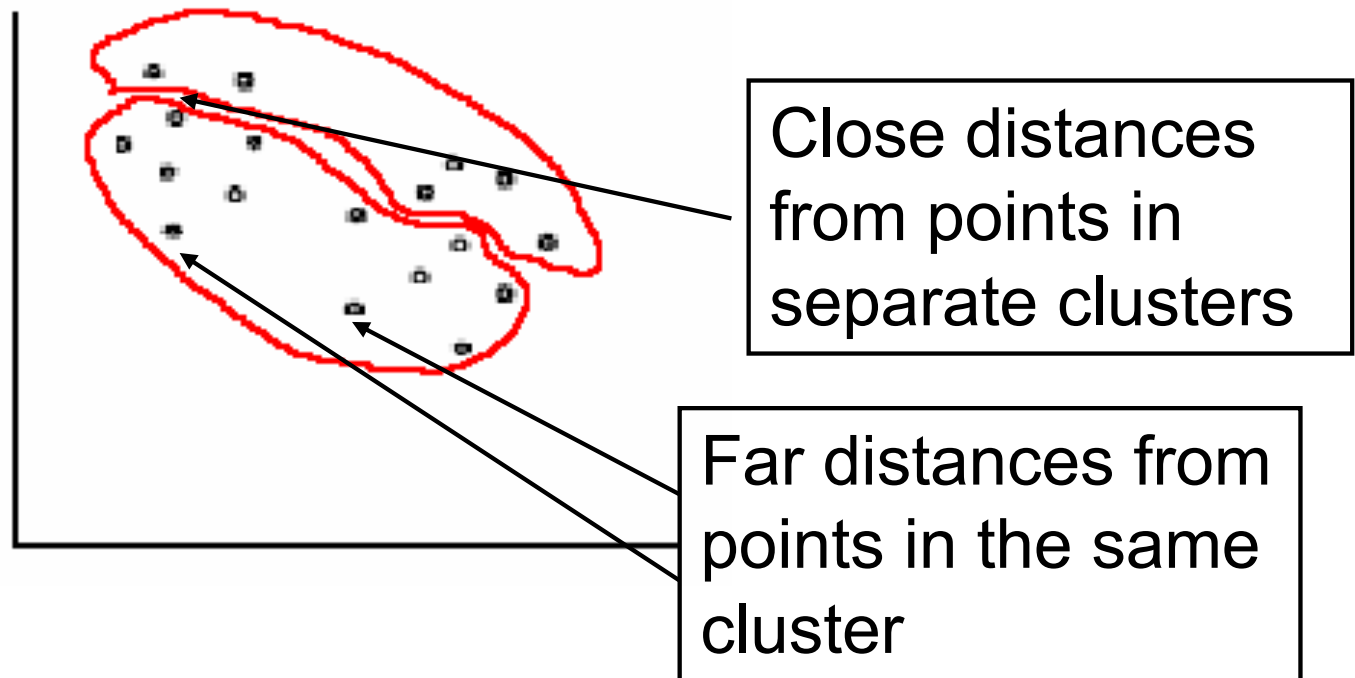
- **Homogeneity:** Elements within a cluster are close to each other
- **Separation:** Elements in different clusters are further apart from each other
- ...clustering is not an easy task!

Given these points a clustering algorithm → might make two distinct clusters as follows



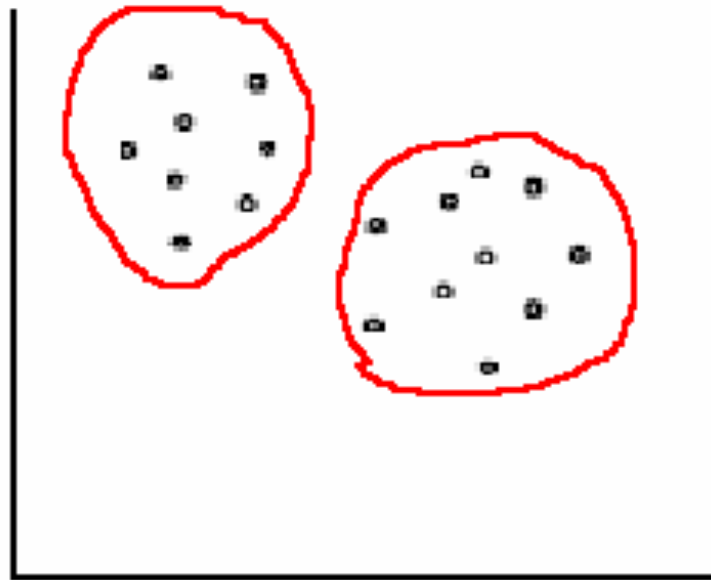
Bad Clustering

This clustering violates both Homogeneity and Separation



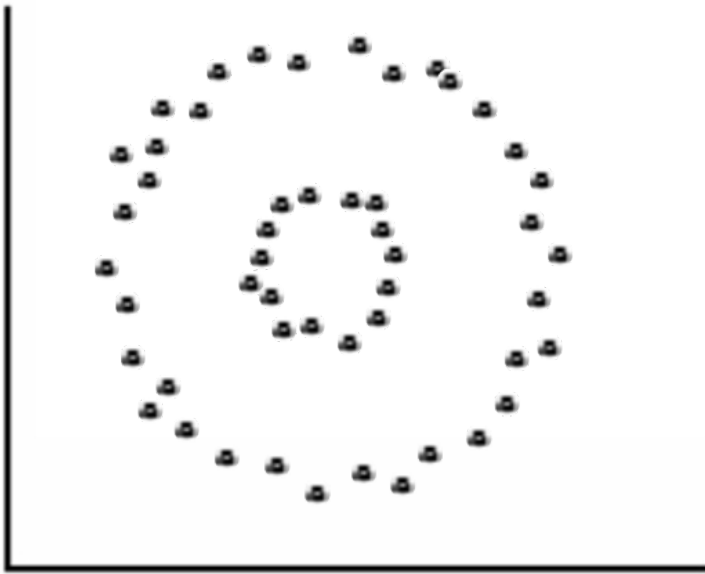
Good Clustering

This clustering exhibits both **good** Homogeneity and Separation



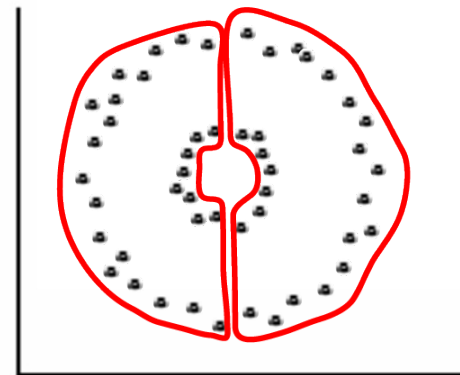
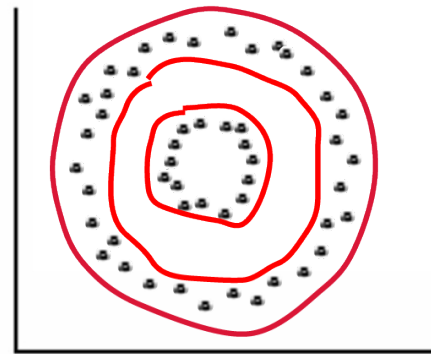
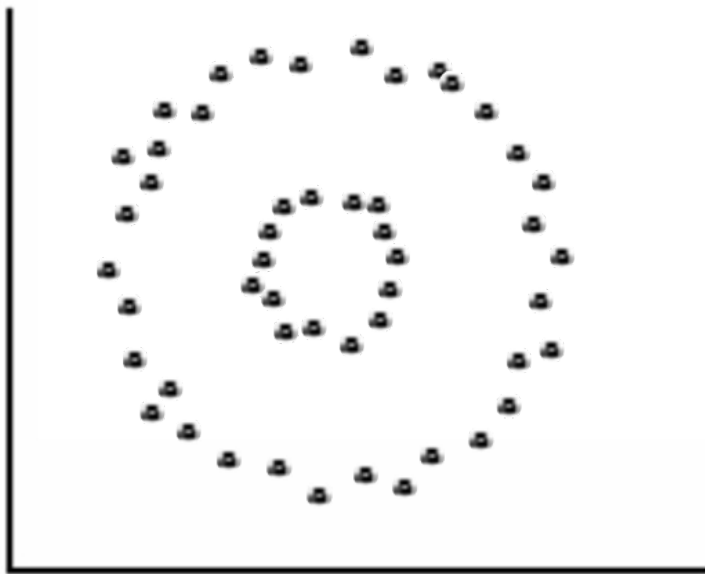
Good clustering (cont.)

What is a good homogeneity and separation here?



Good clustering (cont.)

What is a good homogeneity and separation here?



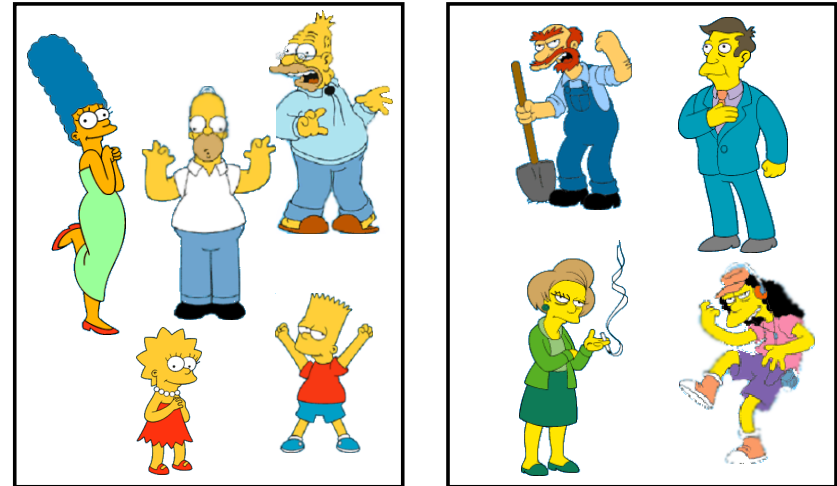
Both cluster solutions have good homogeneity and separation.

How hard is clustering?

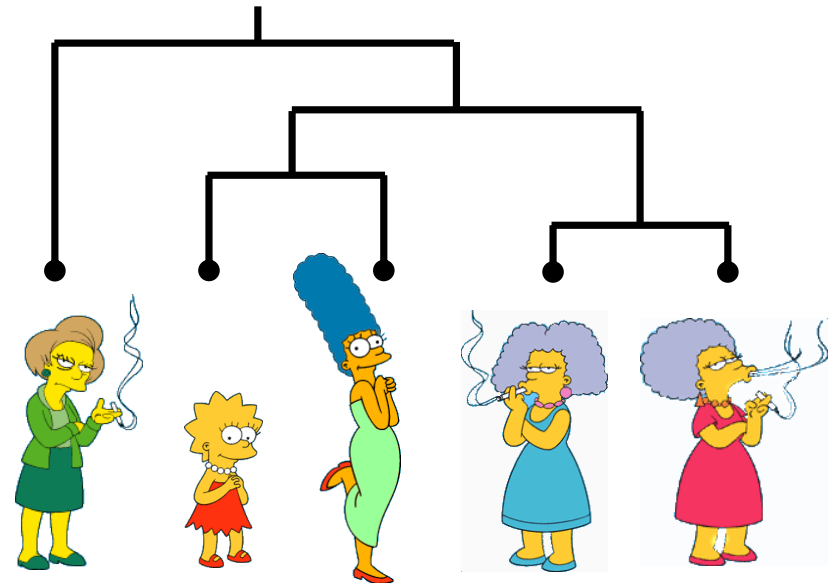
- One idea is to consider all possible clusterings, and pick the one that has the best inter and intra-class similarity.
- Suppose we are given n points and would like to cluster them into k -clusters
 - How many possible clusterings are?
 $O(k^n)$
- Too hard to do brute force

Clustering methods

- Partitioning methods



- Hierarchical methods



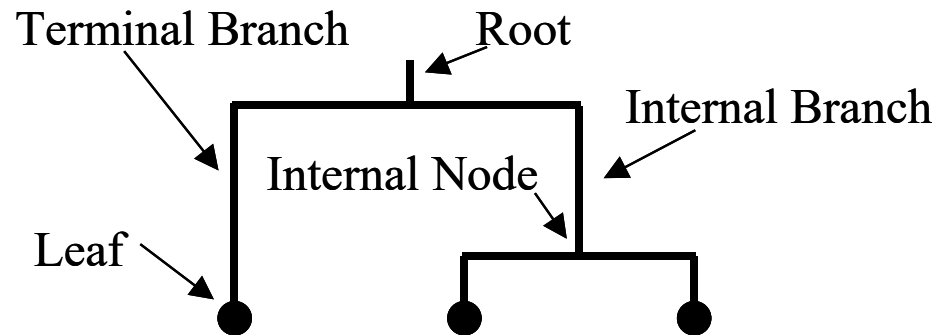
Clustering Techniques

- **Agglomerative:** Start with every element in its own cluster, and iteratively join clusters together
- **Divisive:** Start with one cluster and iteratively divide it into smaller clusters

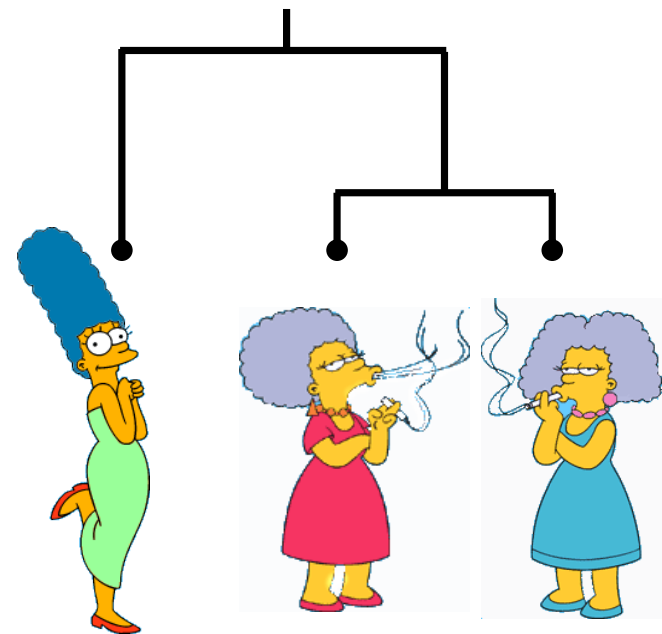
Dendrogram

- A graphical procedure for representing the output of a hierarchical clustering method.
- A binary tree that has all the data items at its leaves.
- The ordering of the leaves is arbitrary, as is their horizontal position.
- The heights of the internal nodes may be arbitrary, or may be related to the metric information used to form the clustering

Dendrogram (cont.)



The similarity between two objects in a dendrogram is represented as the height of the lowest internal node they share.



Hierarchical Clustering

- The number of dendrograms with n leafs = $(2n - 3)! / [(2^{n-2}) (n - 2)!]$

- **Bottom-Up (agglomerative):**

- Starting with each item in its own cluster, find the best pair to merge into a new cluster. Repeat until all clusters are fused together.

Number of Leafs	Number of Possible Dendrograms
2	1
3	3
4	15
5	105
...	...
10	34,459,425

- **Top-Down (divisive):**

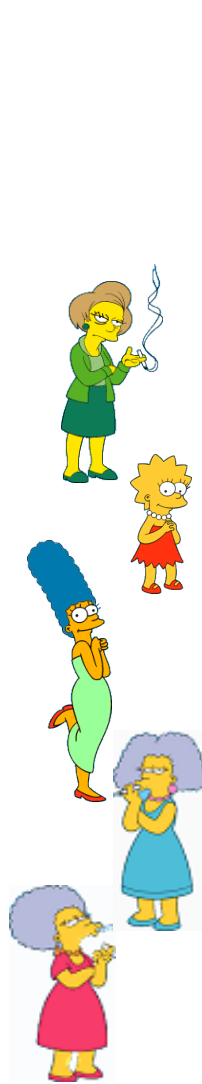
- Starting with all the data in a single cluster, consider every possible way to divide the cluster into two. Choose the best division and recursively operate on both sides.

Hierarchical Clustering: Example 1

We begin with a distance matrix which contains the distances between every pair of objects in our database.


$$D(\text{Marge}, \text{Lisa}) = 8$$

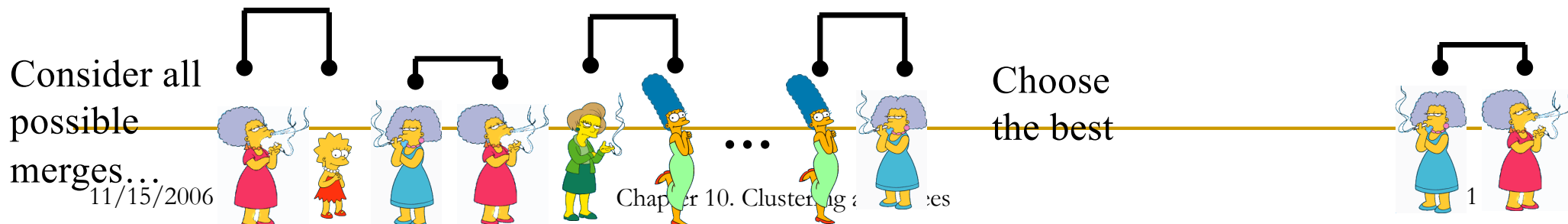

$$D(\text{Auntie Plaut}, \text{Edna Krabappel}) = 1$$



				
0	8	8	7	7
	0	2	4	4
		0	3	3
			0	1
				0
				

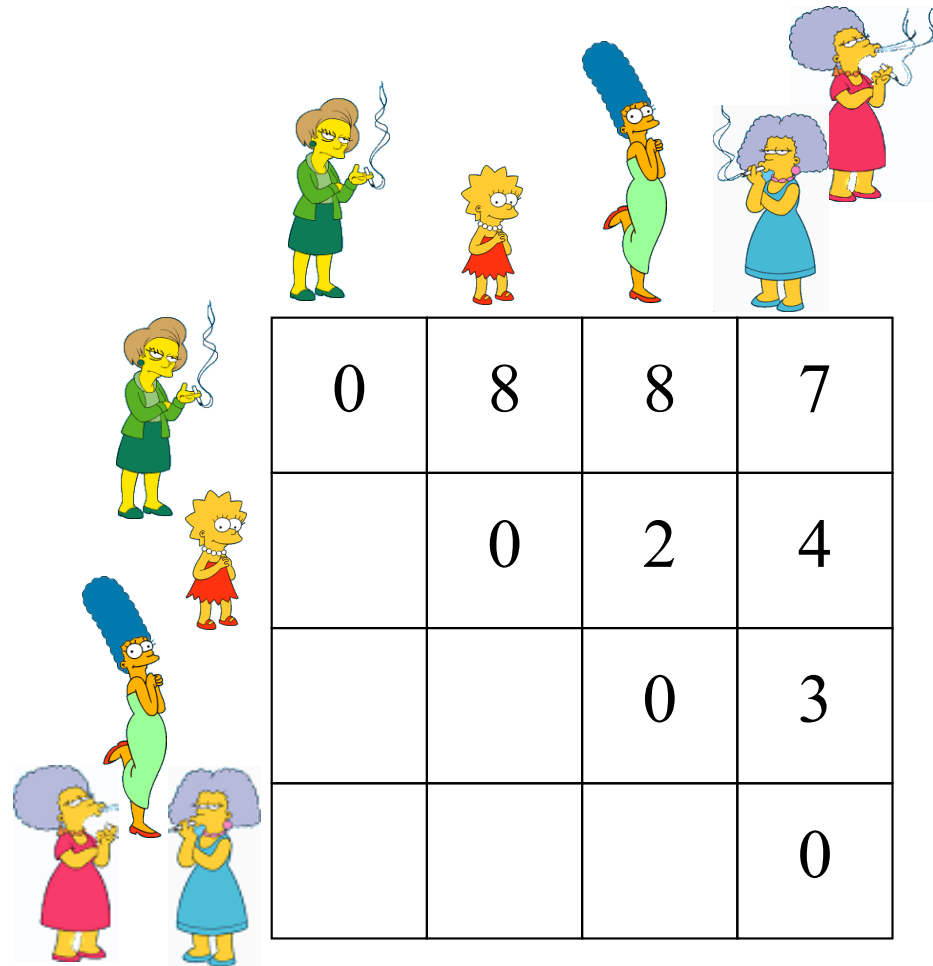
Hierarchical Clustering: Example 1

Bottom-Up (agglomerative): Starting with each item in its own cluster, find the best pair to merge into a new cluster. Repeat until all clusters are fused together.



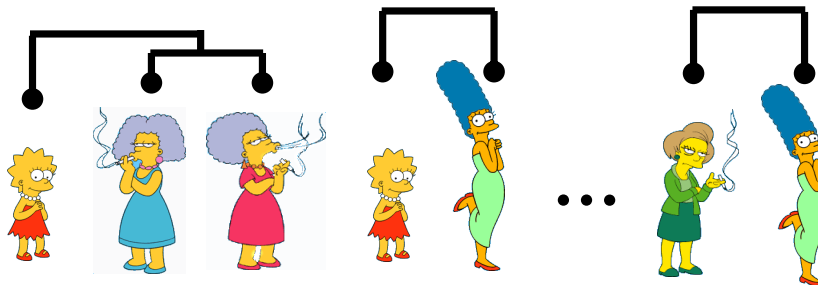
Example 1 (cont'd)

Re-computing distances

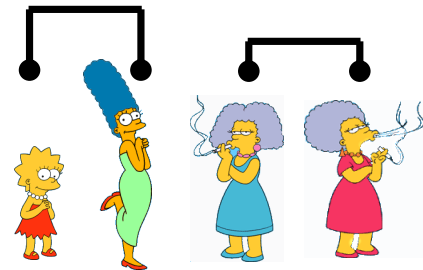


Hierarchical Clustering: Example 1

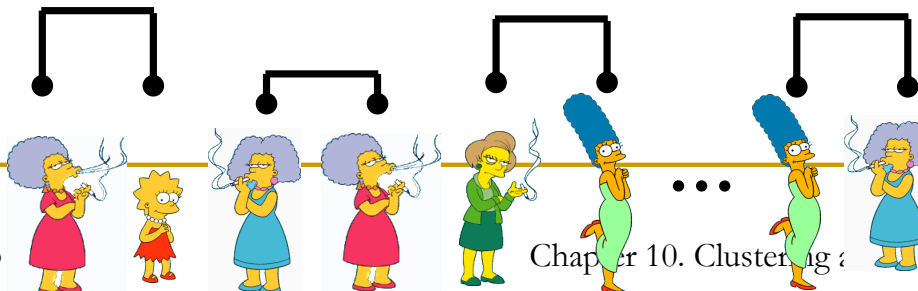
Consider all
possible
merges...



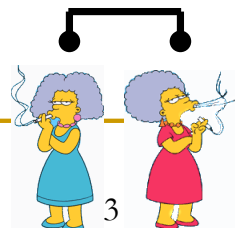
Choose
the best



Consider all
possible
merges...



Choose
the best



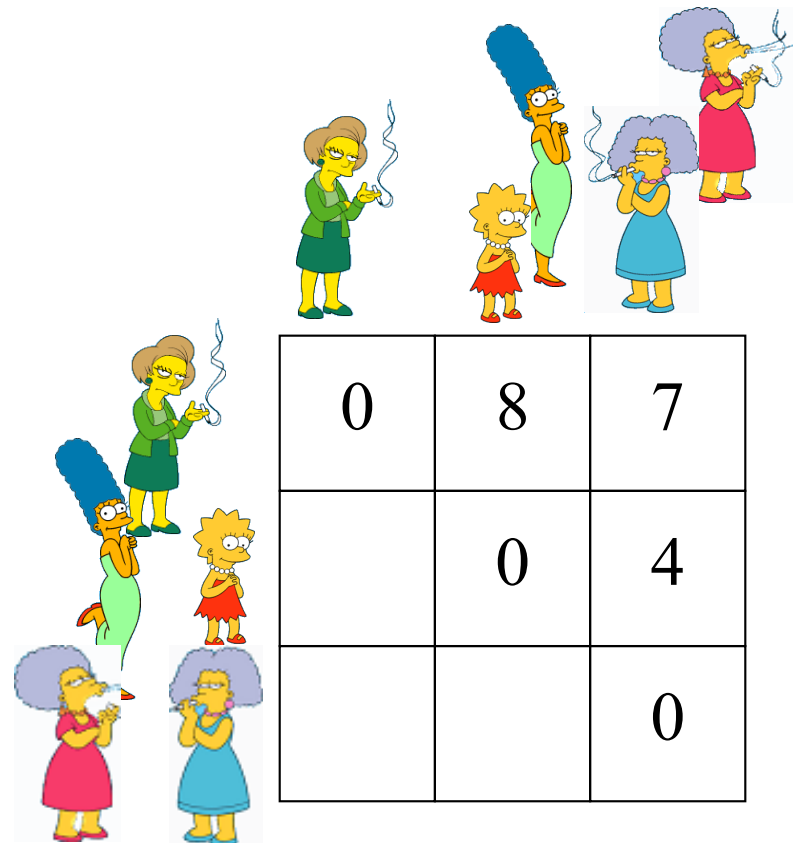
11/15/2006

Chapter 10. Clustering: Hierarchical Clustering

3

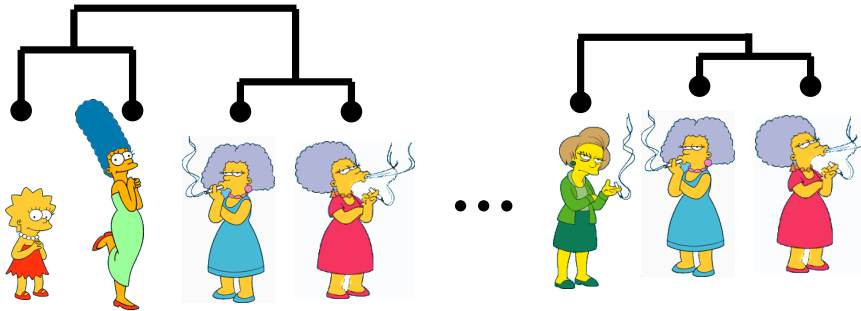
Example 1 (cont'd)

Re-computing distances

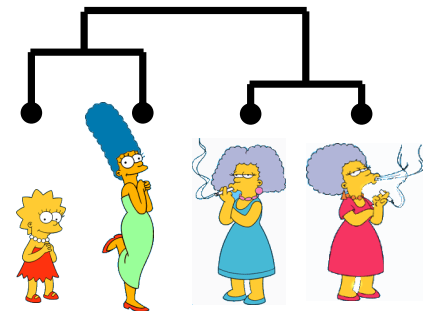


Hierarchical Clustering: Example 1

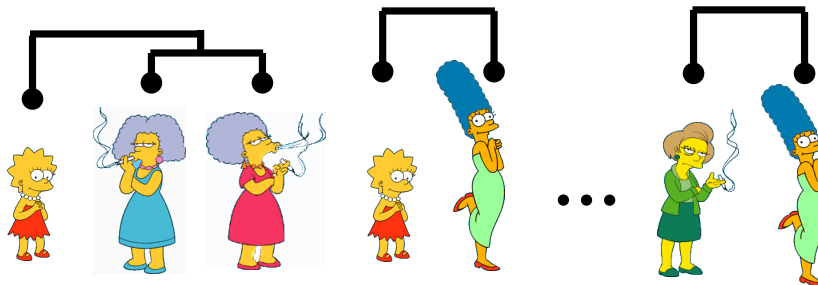
Consider all possible merges...



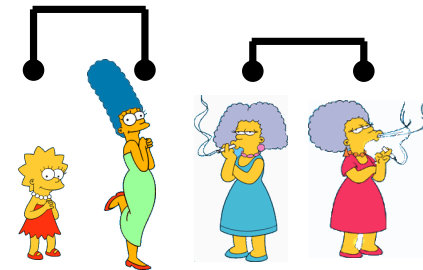
Choose the best



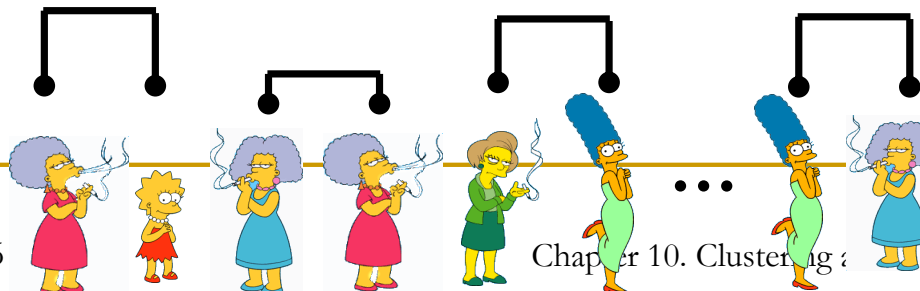
Consider all possible merges...



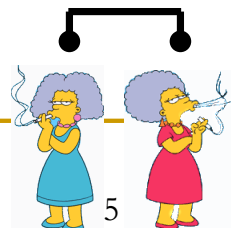
Choose the best



Consider all possible merges...



Choose the best



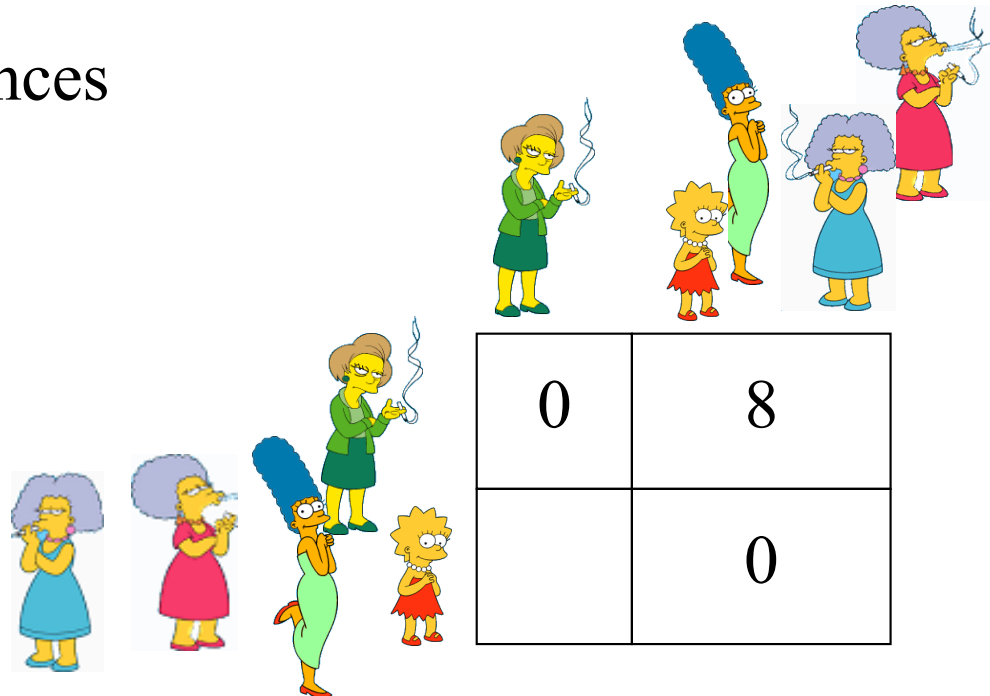
11/15/2006

Chapter 10. Clustering: Hierarchical Clustering

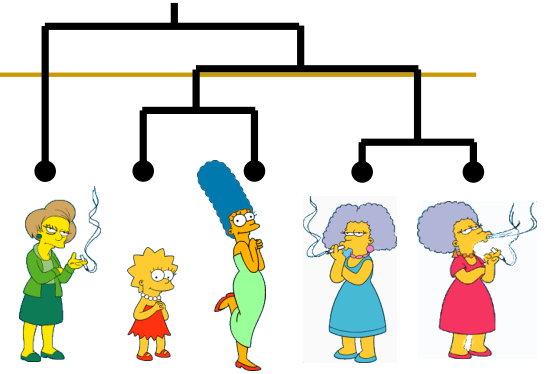
5

Example 1 (cont'd)

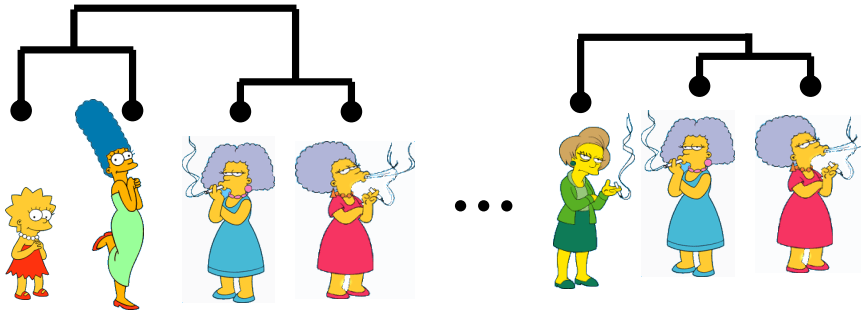
Re-computing distances



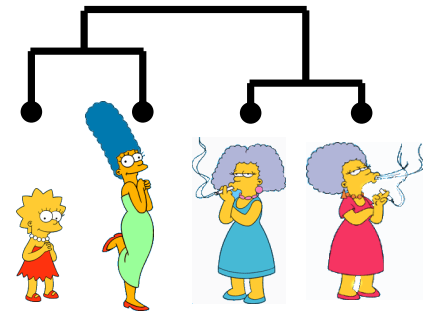
Hierarchical Clustering: Example 1



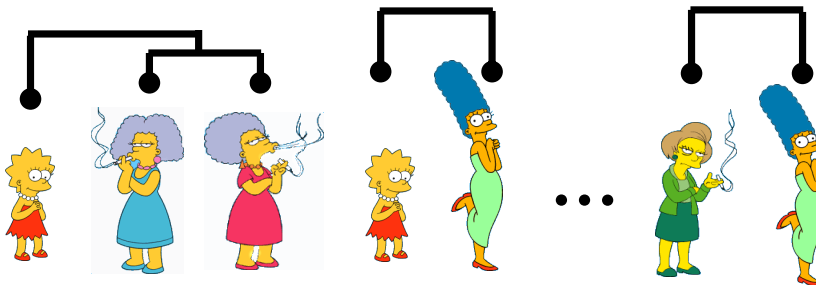
Consider all possible merges...



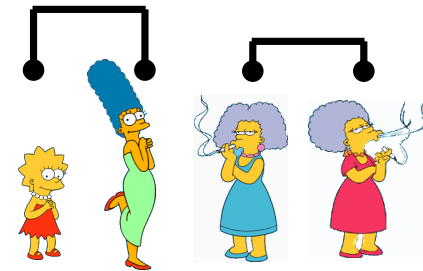
Choose the best



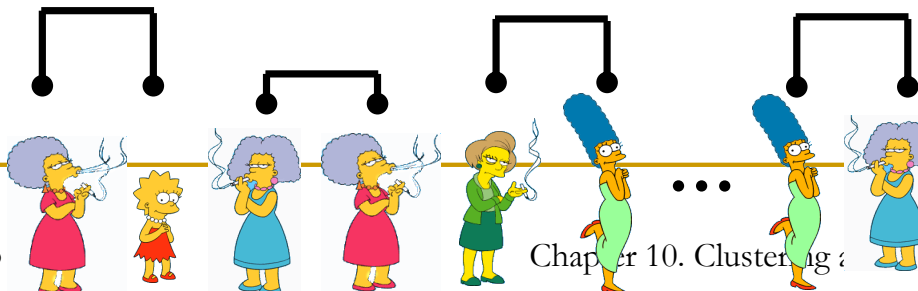
Consider all possible merges...



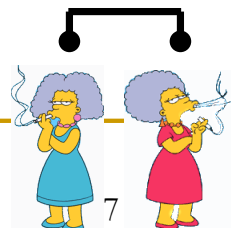
Choose the best



Consider all possible merges...



Choose the best



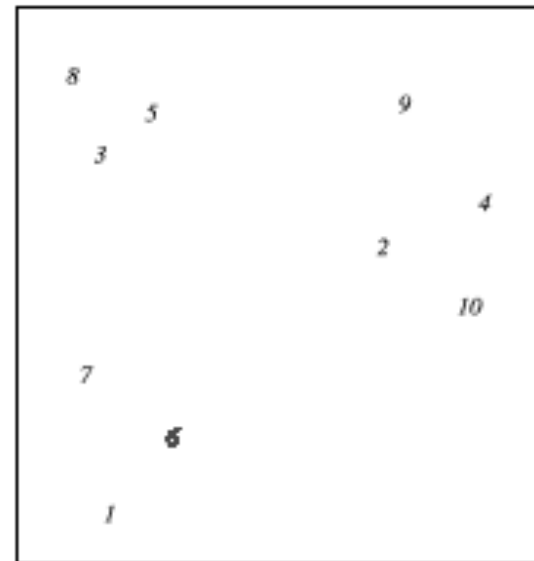
11/15/2006

Chapter 10. Clustering: Exercises

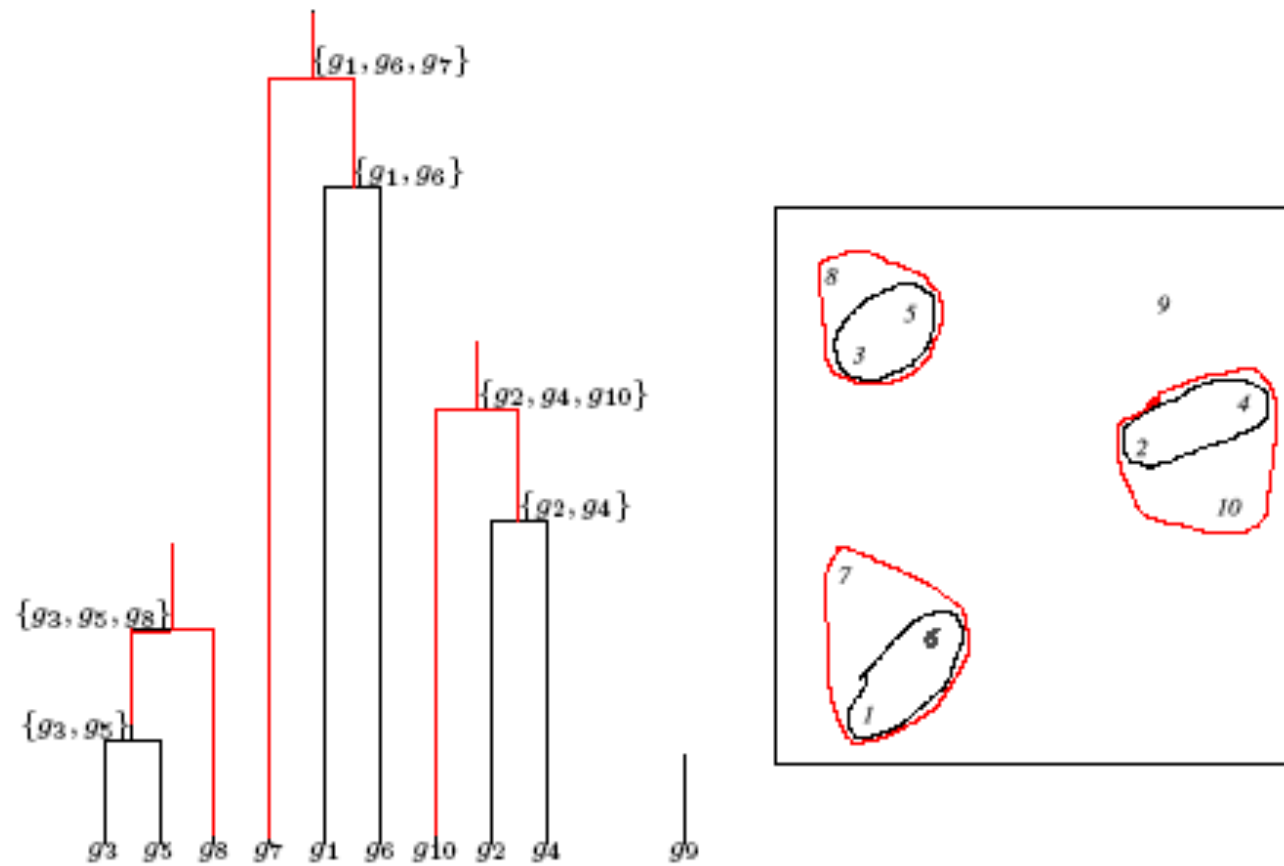
7

Hierarchical Clustering: Example 2

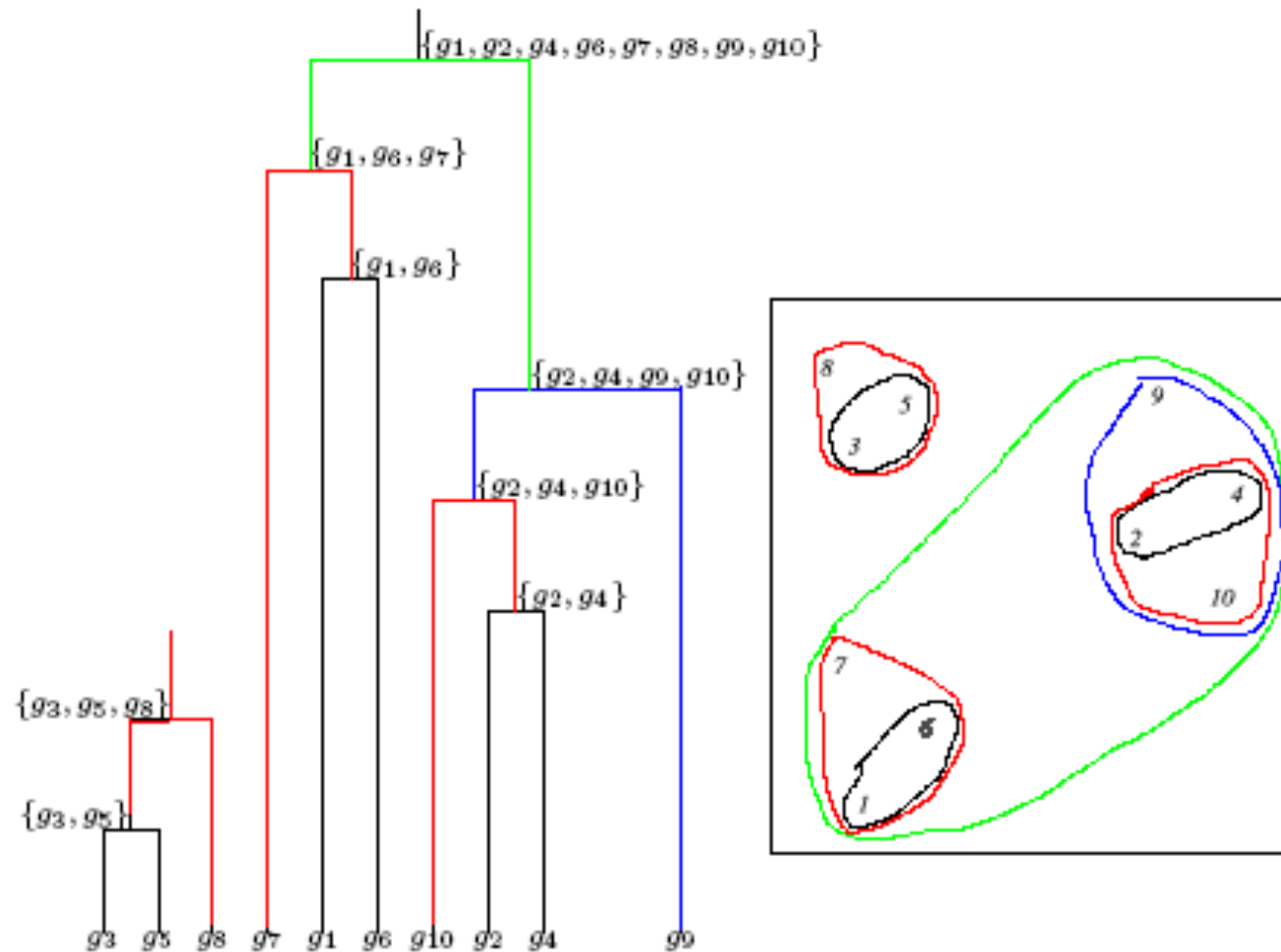
g3 g5 g8 g7 g1 g6 g10 g2 g4 g9



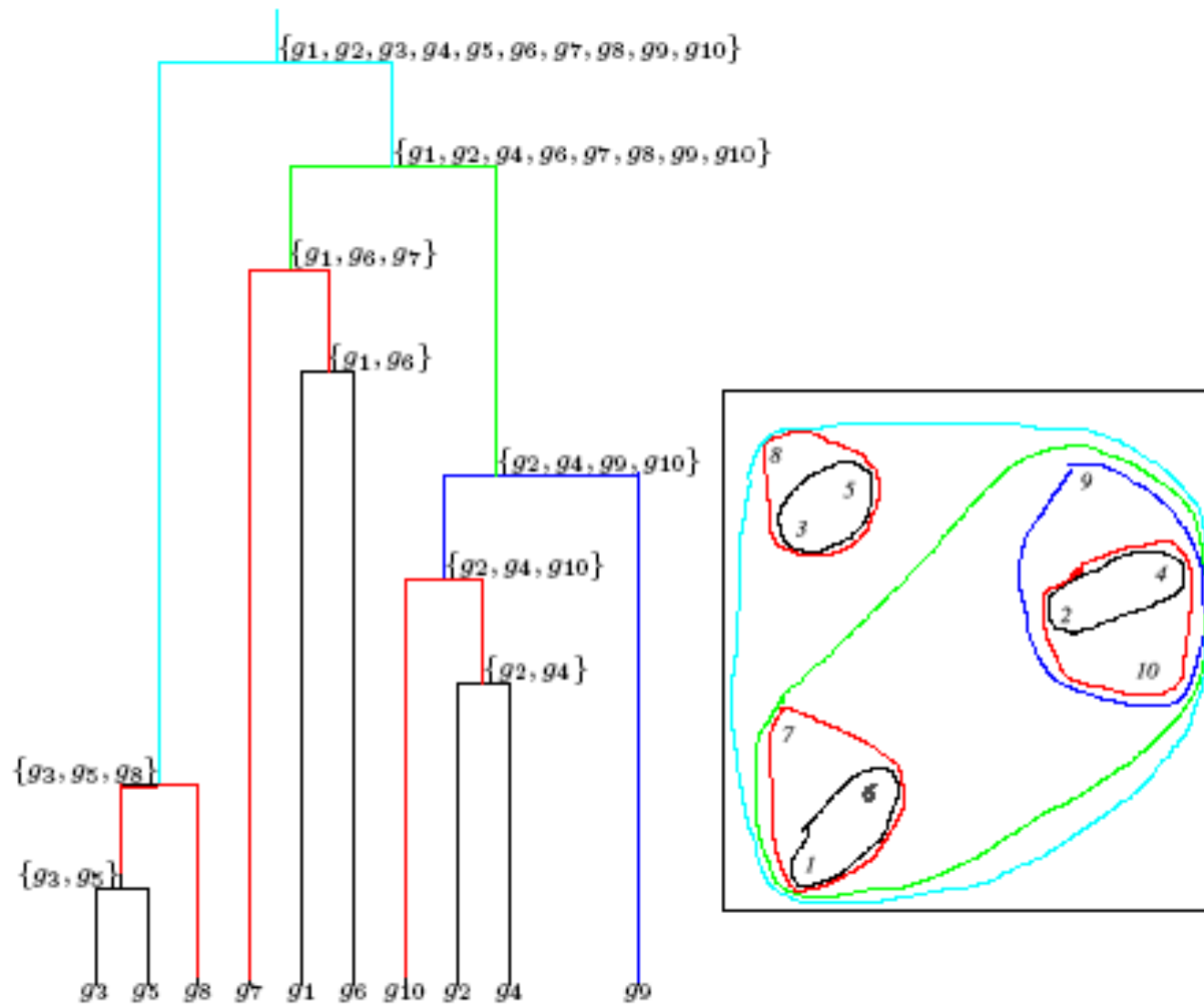
Hierarchical Clustering: Example 2



Hierarchical Clustering: Example 2

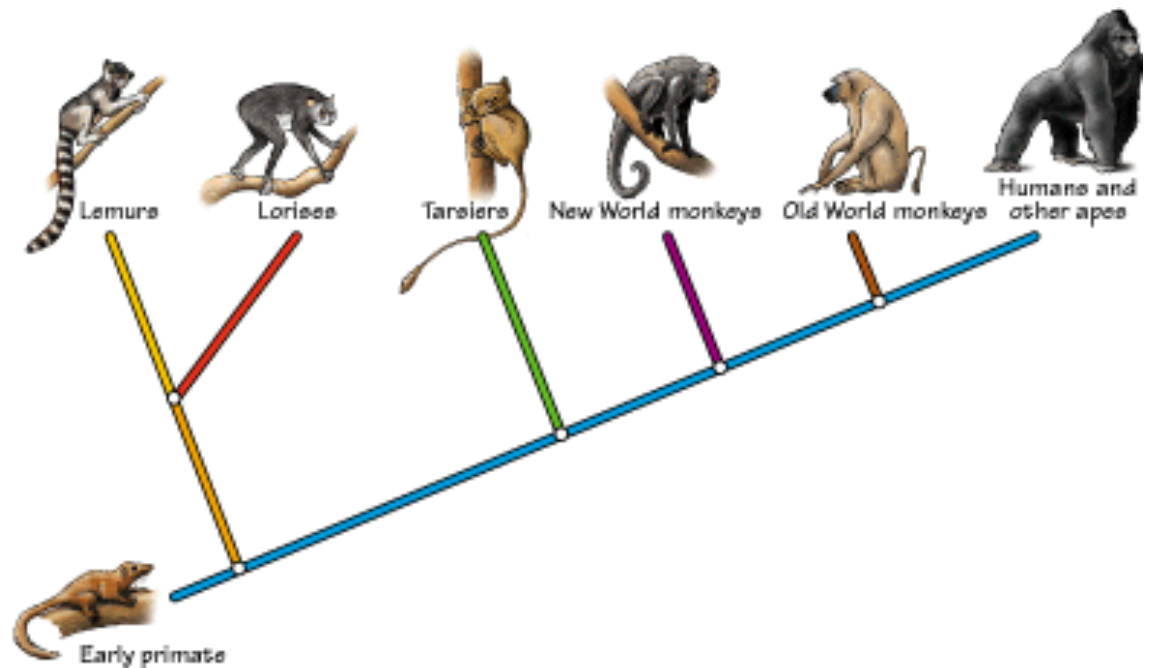


Hierarchical Clustering: Example 2



Hierarchical Clustering

- Hierarchical Clustering is often used to reveal evolutionary history
- Here is an example using the evolution of the primates



Hierarchical Clustering Algorithm

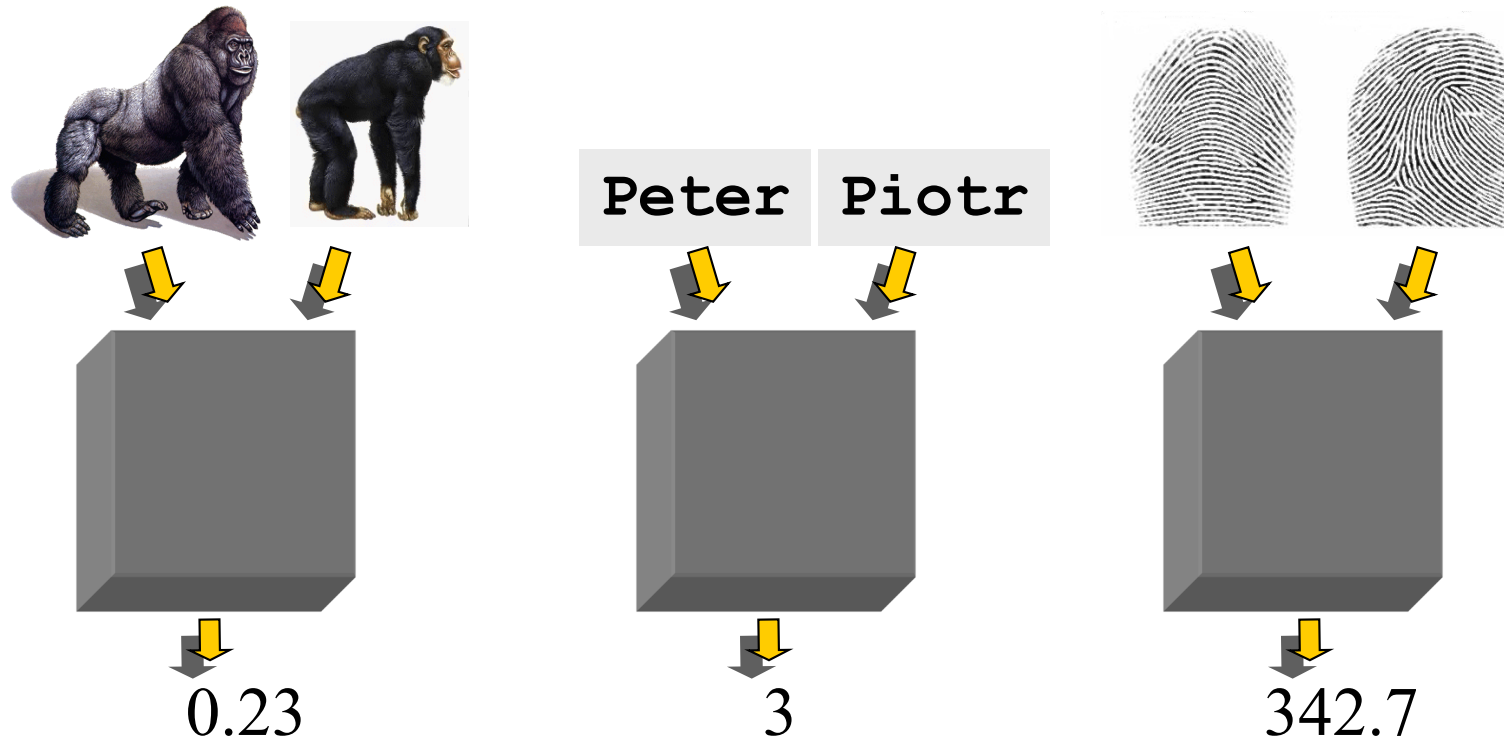
Hierarchical Clustering (d, n)

- Form n clusters each with one element
- Construct a graph T by assigning an one vertex to each cluster
- while there is more than one cluster
- Find the two closest clusters C_1 and C_2
- Merge C_1 and C_2 into new cluster C with $|C_1| + |C_2|$ elements
- **Compute distance from C to all other clusters**
- Add a new vertex C to T and connect to vertices C_1 and C_2
- Remove rows and columns of d corresponding to C_1 and C_2
- Add a row and column for d for the new cluster C
- return T

The algorithm takes a $n \times n$ distance matrix d of pairwise distances between points

Defining distance measures

- Definition:** Let O_1 and O_2 be two objects from the universe of possible objects. The distance (dissimilarity) between O_1 and O_2 is a real number denoted by $D(O_1, O_2)$



What properties should a distance measure have?

- Symmetry
 - $D(A,B) = D(B,A)$
- Constancy of self-similarity
 - $D(A,A) = 0$
- Positivity (separation)
 - $D(A,B) = 0$ IIf $A = B$
- Triangular inequality
 - $D(A,B) \leq D(A,C) + D(B,C)$

Hierarchical Clustering: Recomputing Distances

- Different ways to define distances between points/clusters may lead to different clusterings.
- **Single linkage (nearest neighbor):** In this method the distance between two clusters is determined by the distance of the two closest objects (nearest neighbors) in the different clusters.
 - $d_{\min}(C, C^*) = \min d(x,y)$ for all elements x in C^* and y in C
- **Complete linkage (furthest neighbor):** In this method, the distances between clusters are determined by the greatest distance between any two objects in the different clusters (i.e., by the "furthest neighbors").
 - $d_{\max}(C, C^*) = \max d(x,y)$ for all elements x in C^* and y in C
- **Group average:** In this method, the distance between two clusters is calculated as the average distance between all pairs of objects in the two different clusters.
 - $d_{\text{avg}}(C, C^*) = (1 / |C^*||C|) \sum d(x,y)$ for all elements x in C^* and y in C

Group average distance

Given two disjoint clusters C_i, C_j of sequences,

$$d_{ij} = \frac{1}{|C_i| \times |C_j|} \sum_{\{p \in C_i, q \in C_j\}} d_{pq}$$

Note that if $C_k = C_i \cup C_j$, then distance to another cluster C_l is:

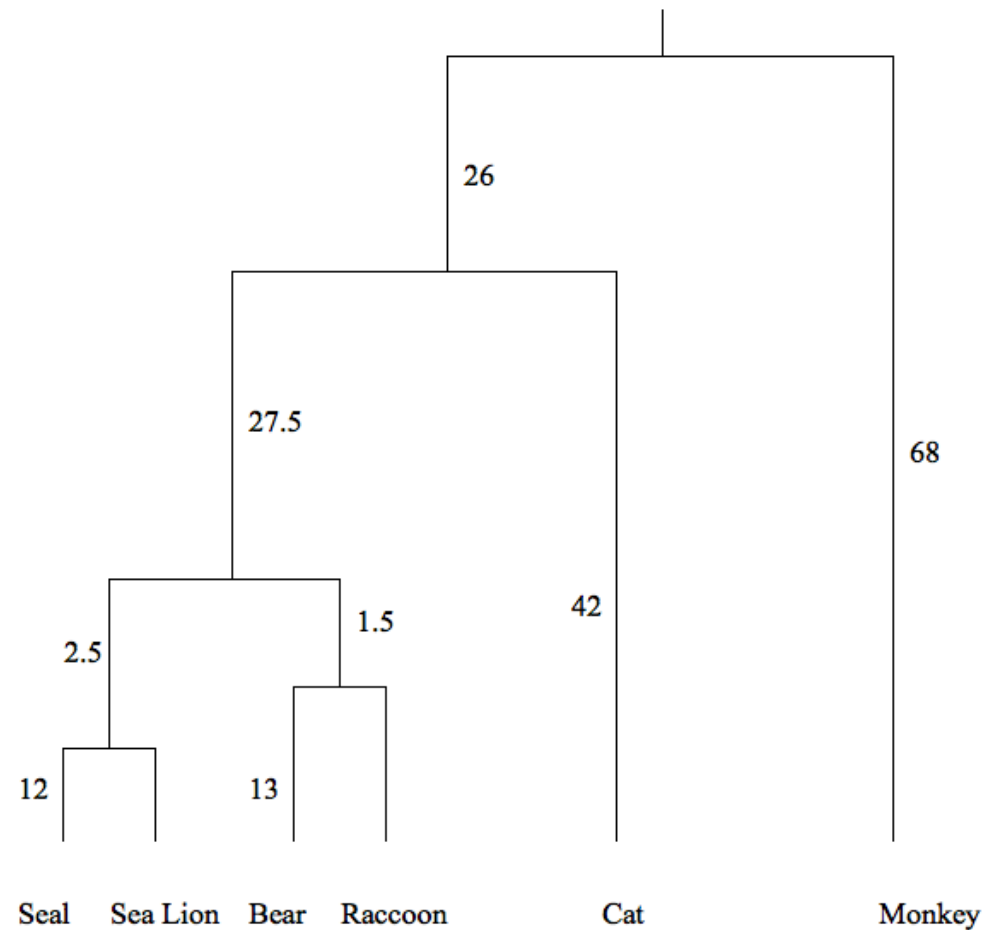
$$d_{kl} = \frac{d_{il} |C_i| + d_{jl} |C_j|}{|C_i| + |C_j|}$$

Hierarchical Clustering- Example

- Consider the distance matrix below.
- Build a hierarchical evolutionary tree from the above distance matrix and using the single linkage (nearest neighbor) method to re-compute distances between clusters.

	Raccoon	Seal	Sea Lion	Cat	Monkey
Bear	26	29	33	84	136
Raccoon		44	44	92	152
Seal			24	89	142
Sea lion				90	142
Cat					148

Hierarchical Clustering- Example



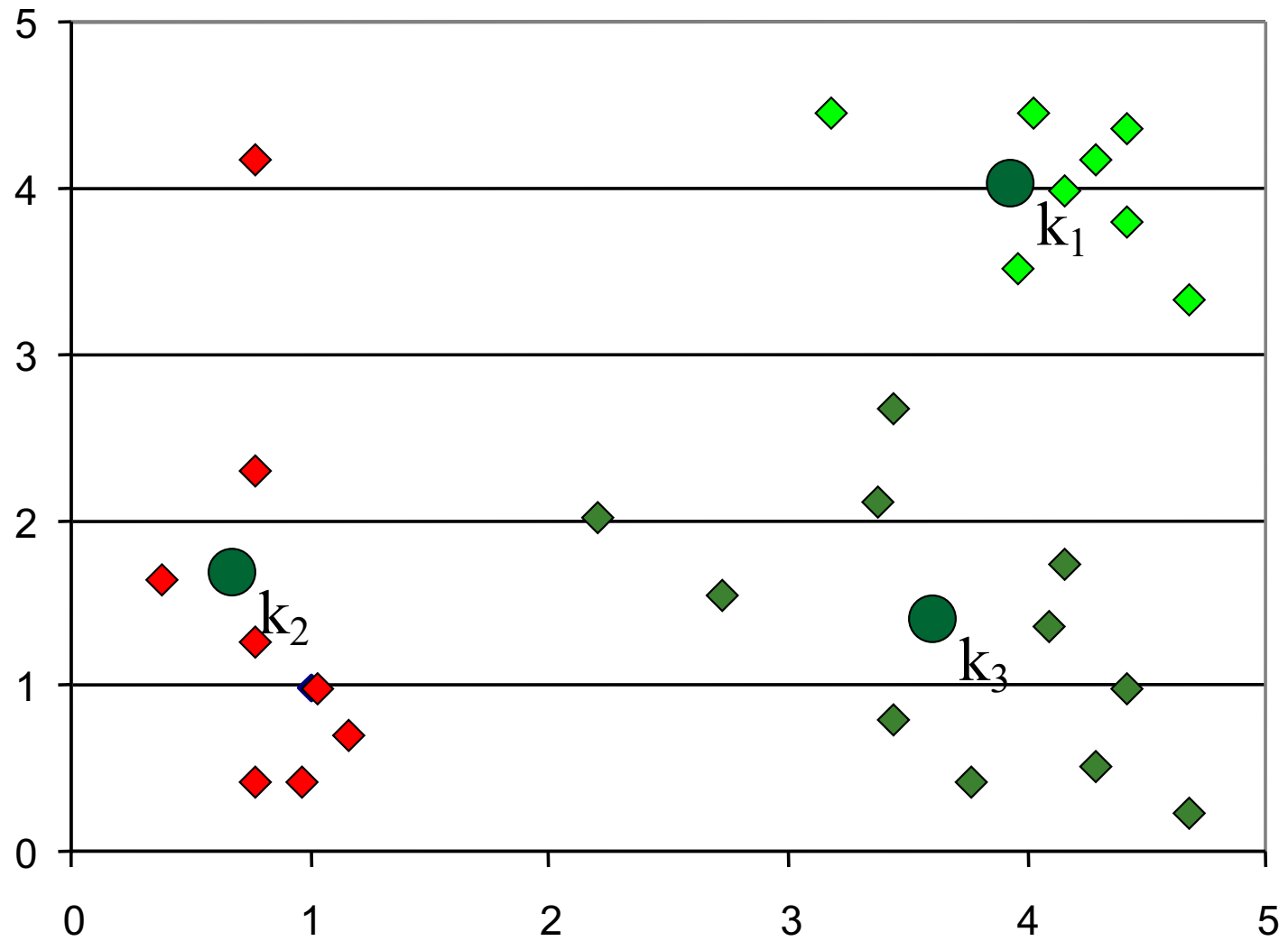
Partitioning Clustering

- K-Means Clustering
 - Configure **K** clusters to enclose all the data points, which minimizes the mean squared distance from each data point to its cluster center, or $d(\mathbf{V}, \mathbf{X})$:

Squared Distortion Error

$$d(\mathbf{V}, \mathbf{X}) = \sum d(v_i, \mathbf{X})^2 / n \quad 1 \leq i \leq n$$

- Note $d(v_i, \mathbf{X})$ refers to Euclidean Distance between the data point v_i and the center of gravity of the corresponding cluster



K-Means Clustering Problem: Formulation

- **Input:** A set, $\mathbf{V}$, consisting of n points and a parameter k
- **Output:** A set $\mathbf{X}$ consisting of k points (cluster centers) that minimizes $d(\mathbf{V}, \mathbf{X})$ over all possible choices of $\mathbf{X}$

This problem is NP-complete.

An efficient heuristic method for K-Means clustering is the Lloyd algorithm

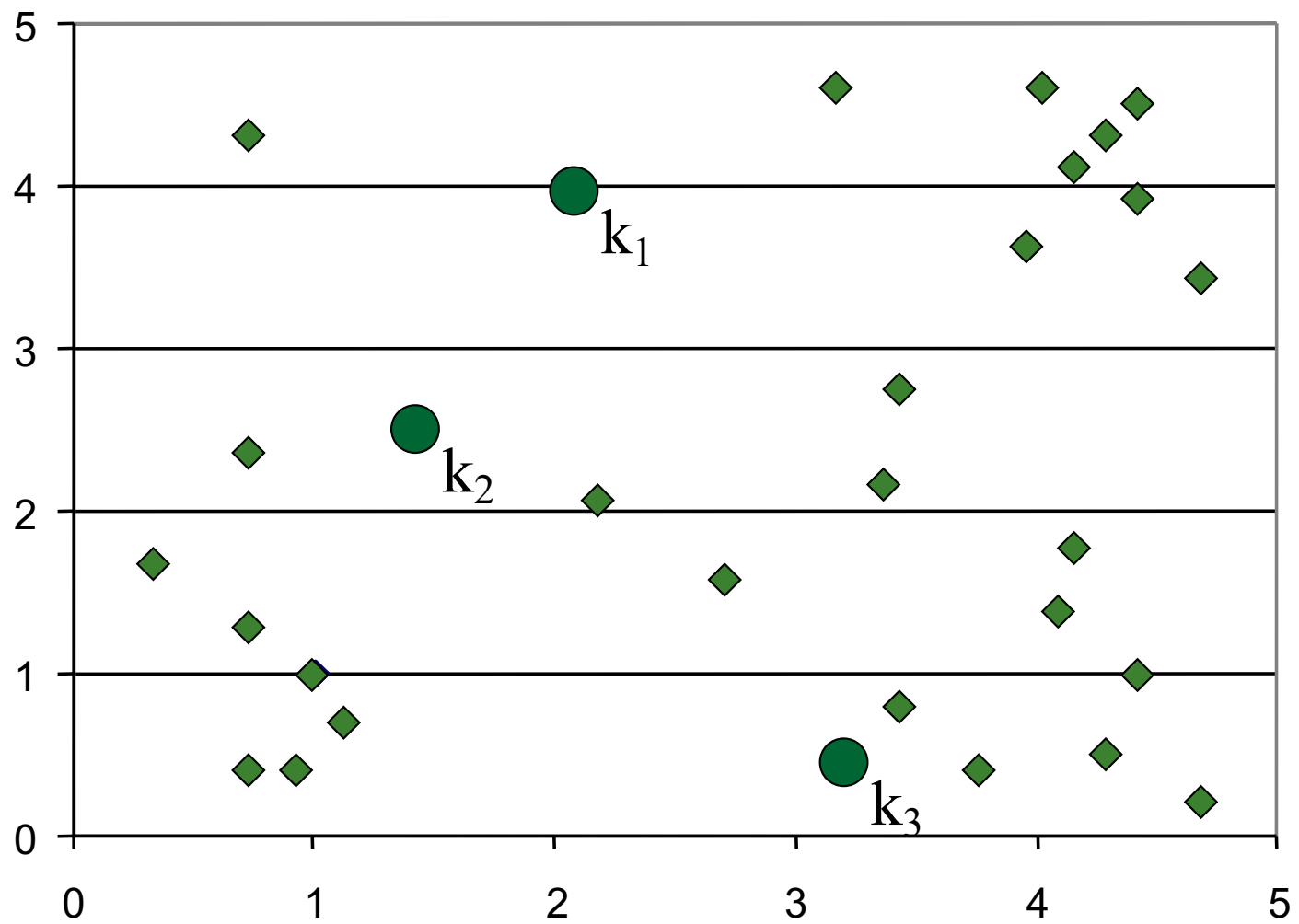
K-Means Clustering: Lloyd Algorithm

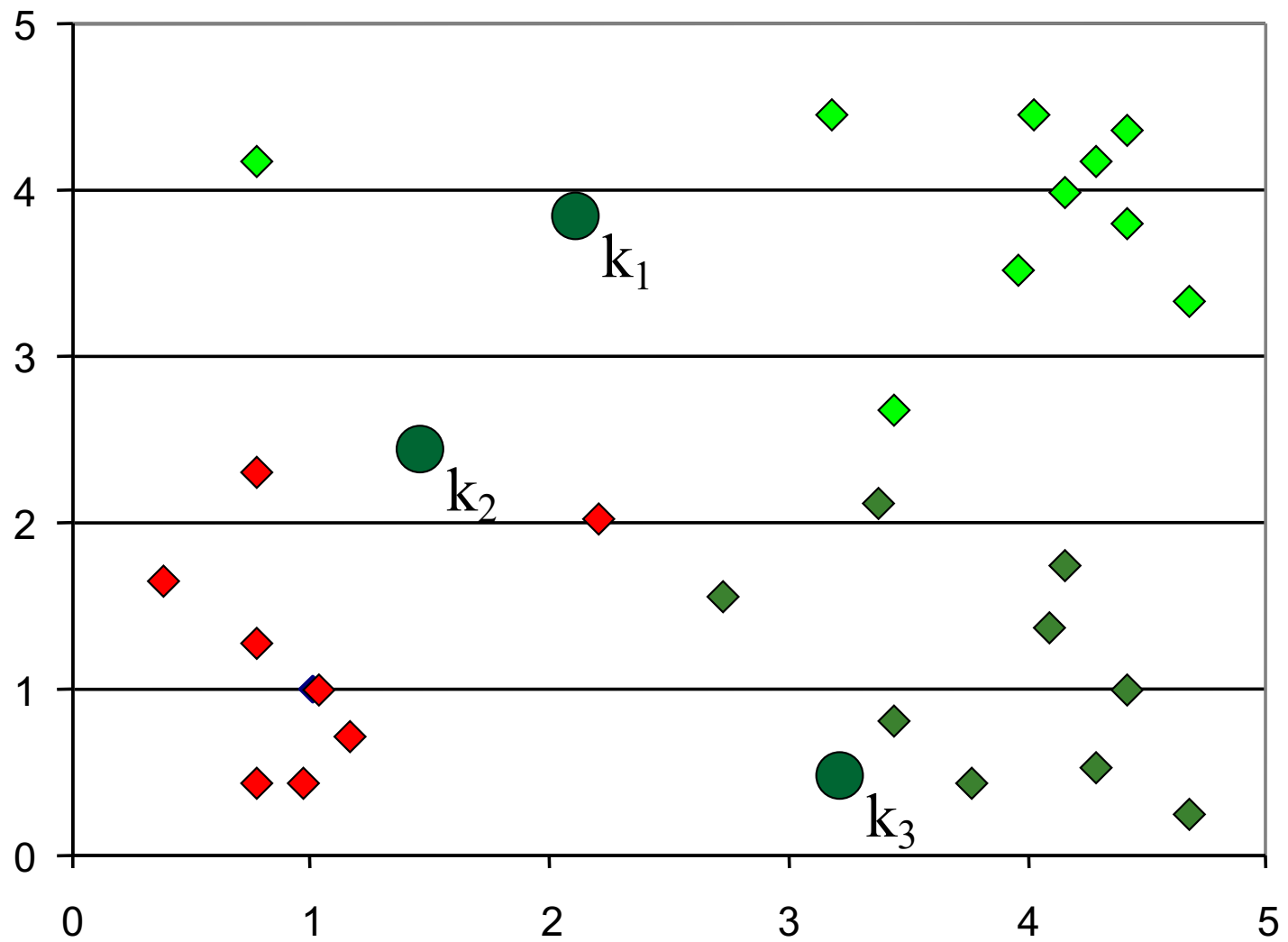
Lloyd Algorithm

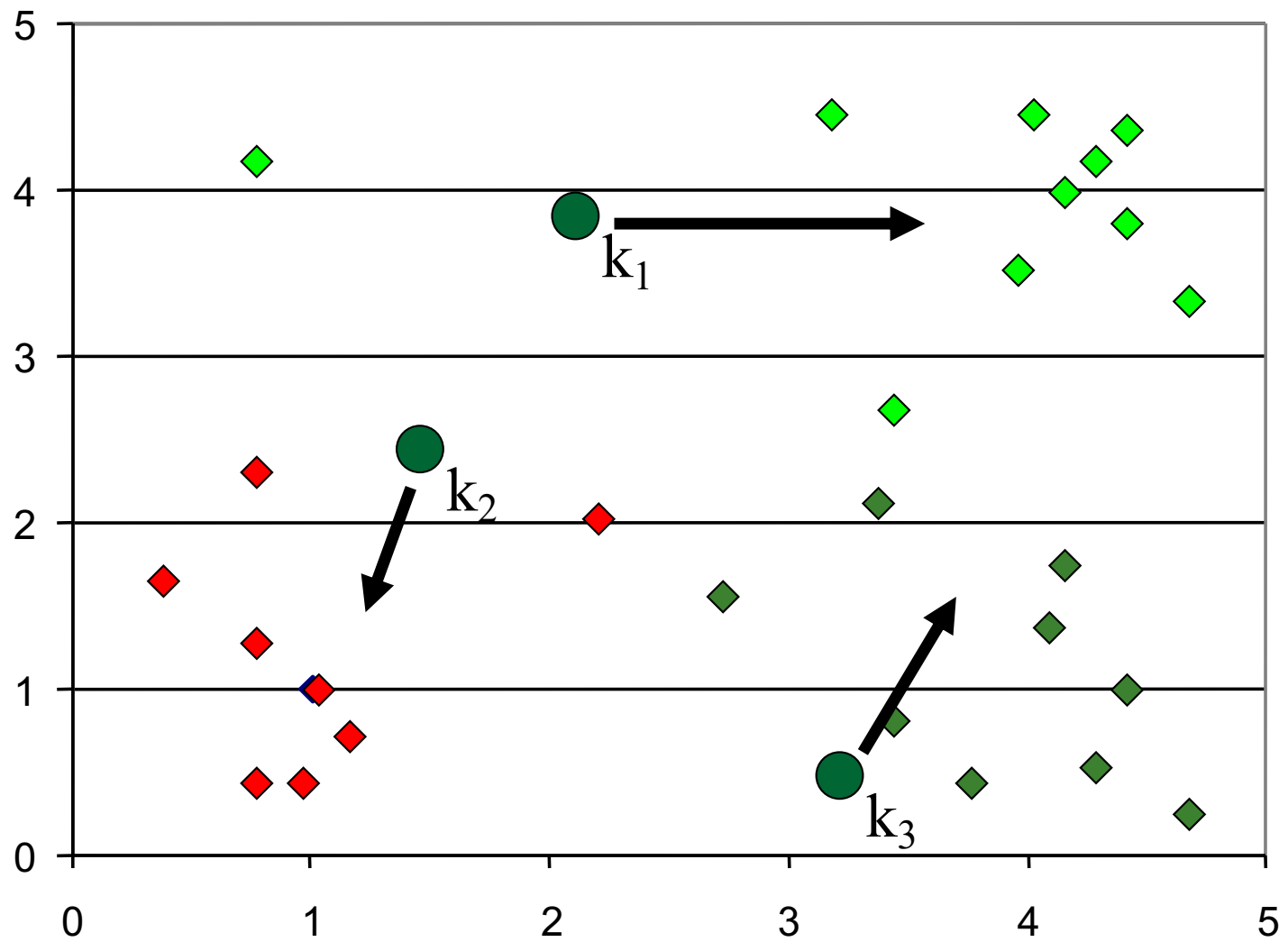
1. Arbitrarily assign the k cluster centers
2. **while** the cluster centers keep changing
3. Assign each data point to the cluster C_i corresponding to the closest cluster representative (center) x_i ($1 \leq i \leq k$)
4. After the assignment of all n data points, compute new cluster representatives according to the center of gravity of each existing cluster, that is, the new cluster representative is

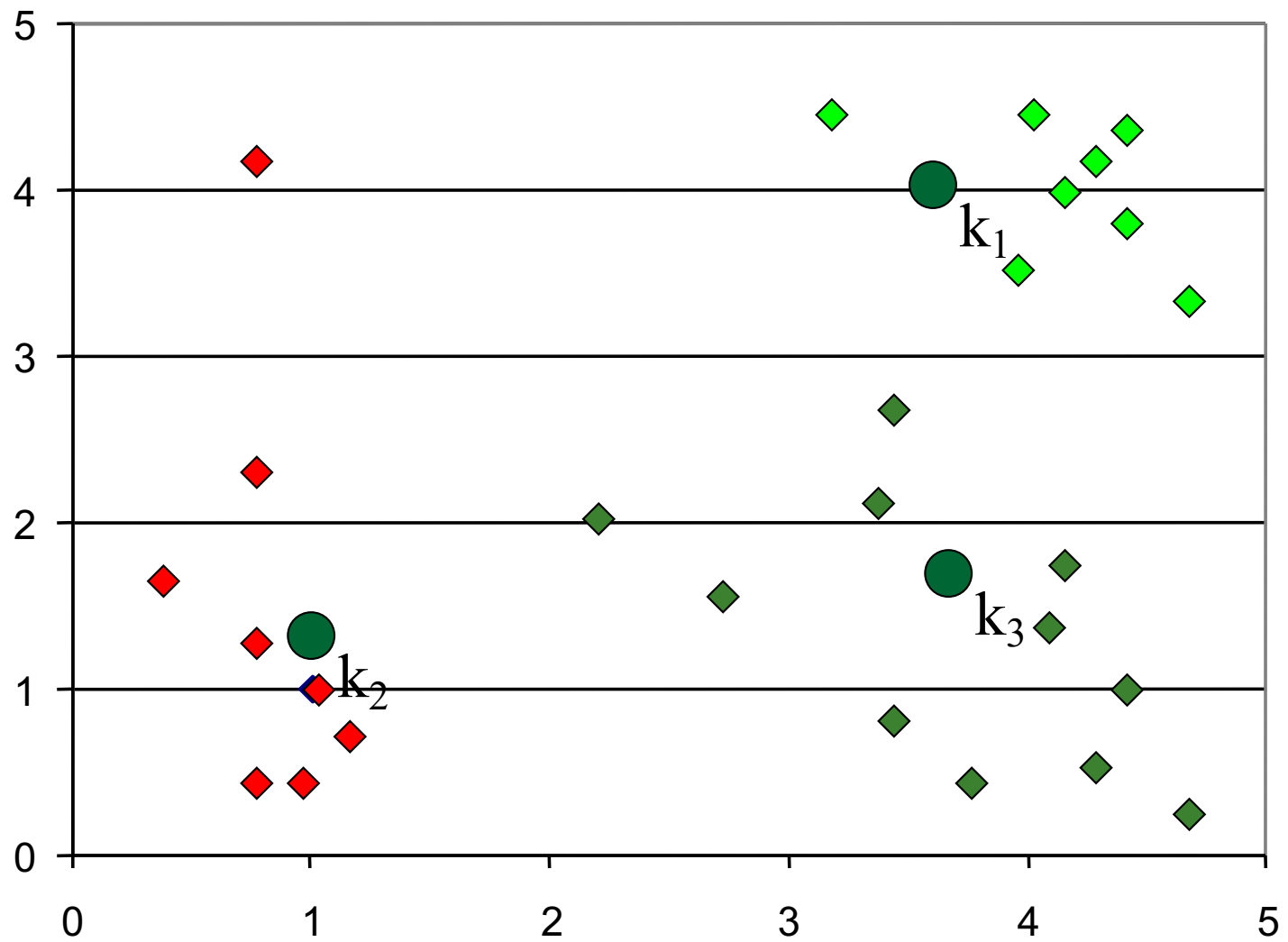
$$\frac{\sum_{v \in C} v}{|C|} \quad \text{for every cluster } C$$

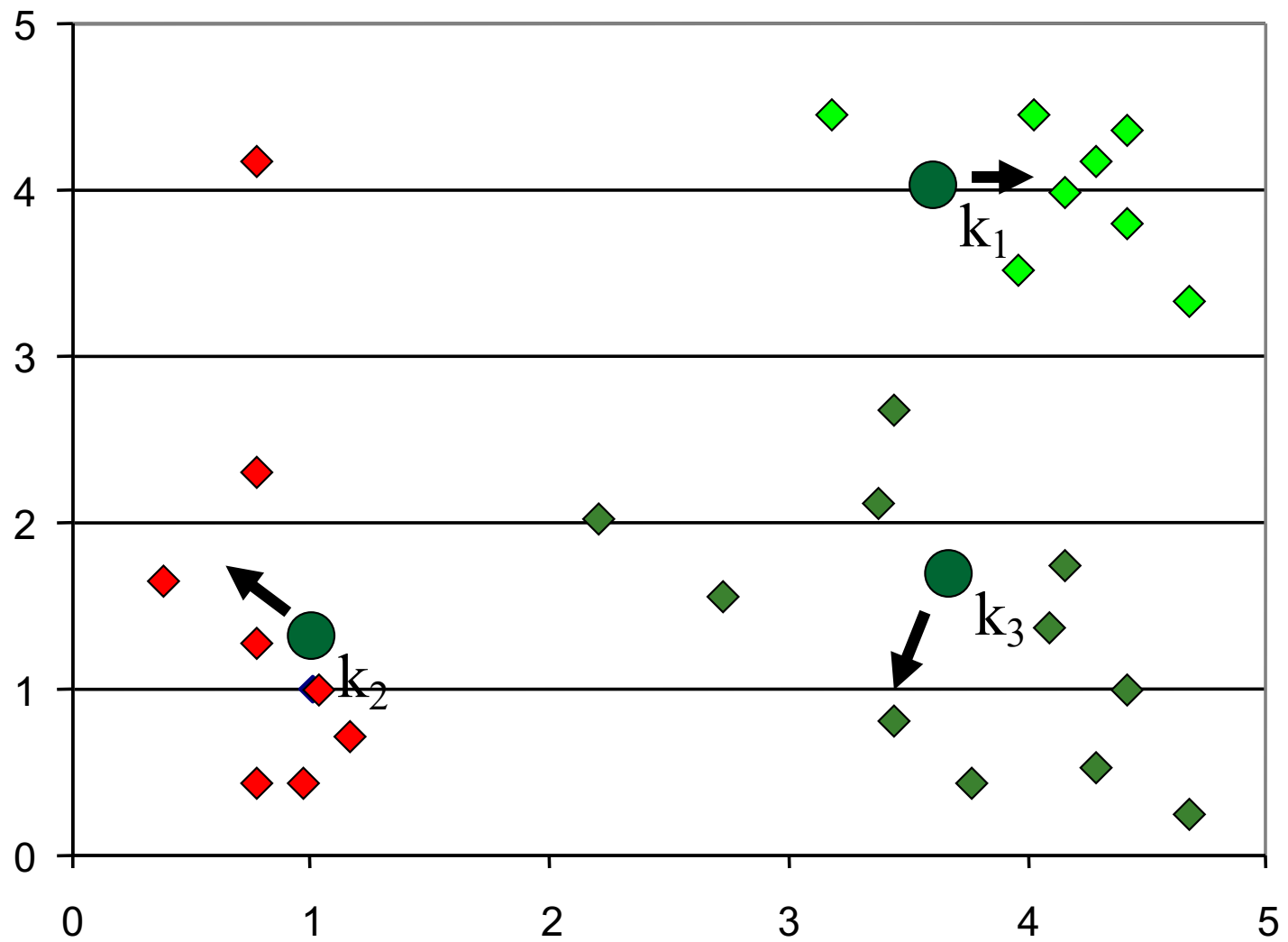
*This may lead to merely a locally optimal clustering.

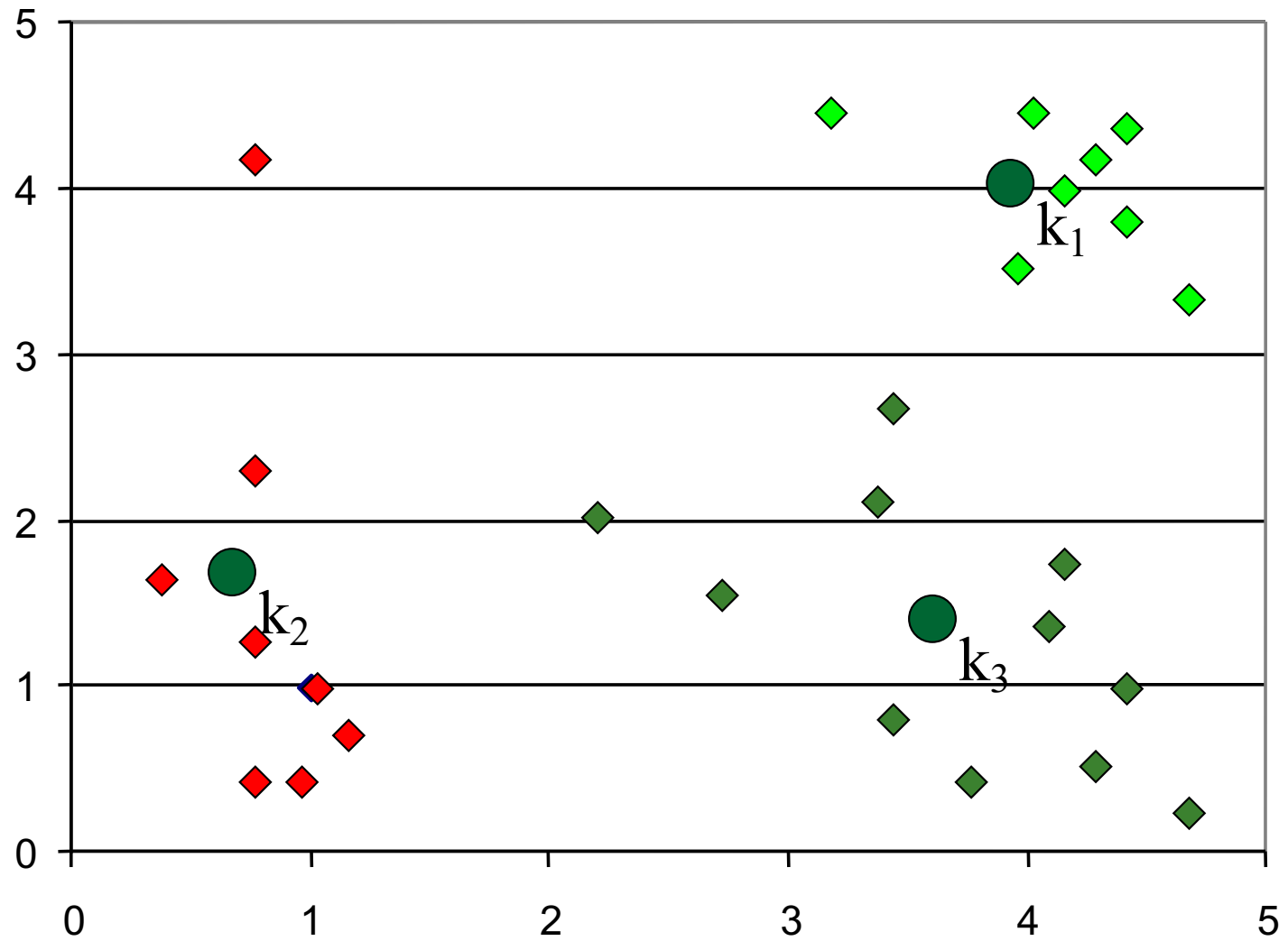












Conservative K-Means Algorithm

- Lloyd algorithm is fast but in each iteration it moves many data points, not necessarily causing better convergence.
- A more conservative method would be to move one point at a time only if it improves the overall clustering cost
 - The smaller the clustering cost of a partition of data points is the better that clustering is
 - Different methods can be used to measure this cost (for example in the last algorithm the squared means was used)

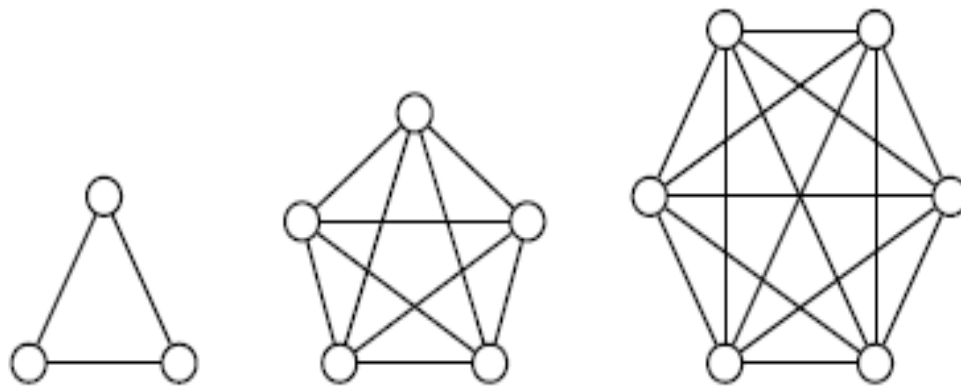
K-Means “Greedy” Algorithm

1. ProgressiveGreedyK-Means(k)
2. Select an arbitrary partition P into k clusters
3. **while** forever
4. $bestChange \leftarrow 0$
5. **for** every cluster C
6. **for** element i not in C
7. **if** moving i to cluster C reduces its clustering cost
8. **if** $cost(P) - cost(P_{i \rightarrow C}) > bestChange$
9. $bestChange \leftarrow cost(P) - cost(P_{i \rightarrow C})$
10. $i^* \leftarrow i$
11. $C^* \leftarrow C$
12. **if** $bestChange > 0$
13. Change partition P by moving i^* to C^*
14. **else**
15. **return** P

Clustering and corrupted cliques

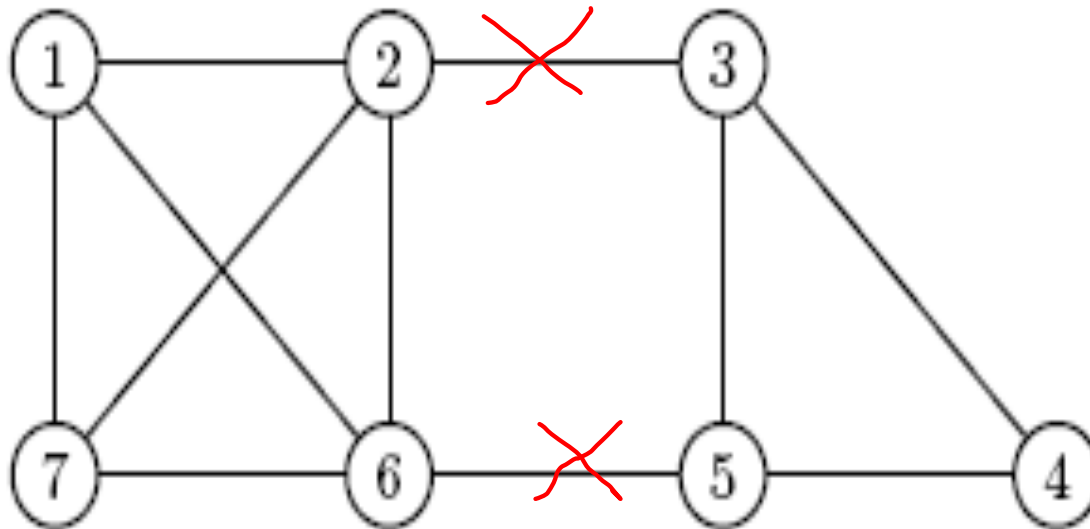
Clique graphs

- A clique is a special type of graph, where every vertex is connected to every other vertex
- A clique graph is a graph where each connected component is a clique



Clique Graphs (cont' d)

- A graph can be transformed into a clique graph by adding or removing edges
 - Example: removing two edges to make a clique graph



Distance Graphs

- Turn the distance matrix into a distance graph
- Choose a distance threshold θ
- Genes are represented as vertices in the graph
- If the distance between two vertices is below θ , draw an edge between them
- The resulting graph may contain cliques
- These cliques represent clusters of data points!

Corrupted Cliques Problem

Input: A graph G

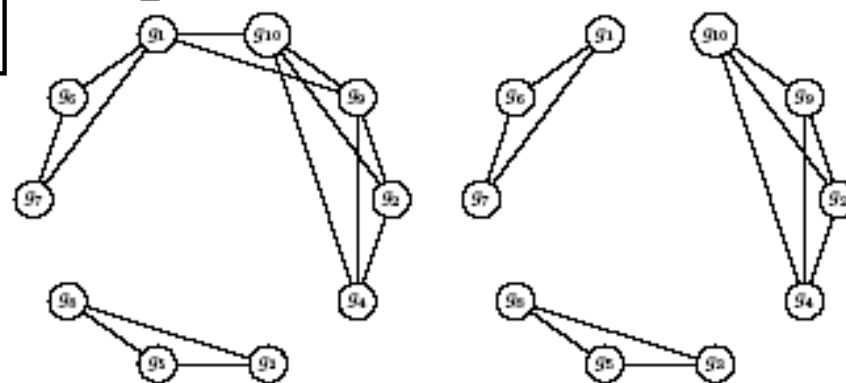
Output: The smallest number of additions and removals of edges that will transform G into a clique graph

Transforming Distance Graph into Clique Graph

The distance graph G (created with a threshold $\theta=7$) is transformed into a clique graph after removing the two highlighted edges

	g_1	g_2	g_3	g_4	g_5	g_6	g_7	g_8	g_9	g_{10}
g_1	0.0	8.1	9.2	7.7	9.3	2.3	5.1	10.2	6.1	7.0
g_2	8.1	0.0	12.0	0.9	12.0	9.5	10.1	12.8	2.0	1.0
g_3	9.2	12.0	0.0	11.2	0.7	11.1	8.1	1.1	10.5	11.5
g_4	7.7	0.9	11.2	0.0	11.2	9.2	9.5	12.0	1.6	1.1
g_5	9.3	12.0	0.7	11.2	0.0	11.2	8.5	1.0	10.6	11.6
g_6	2.3	9.5	11.1	9.2	11.2	0.0	5.6	12.1	7.7	8.5
g_7	5.1	10.1	8.1	9.5	8.5	5.6	0.0	9.1	8.3	9.3
g_8	10.2	12.8	1.1	12.0	1.0	12.1	9.1	0.0	11.4	12.4
g_9	6.1	2.0	10.5	1.6	10.6	7.7	8.3	11.4	0.0	1.1
g_{10}	7.0	1.0	11.5	1.1	11.6	8.5	9.3	12.4	1.1	0.0

(a) Distance matrix, d (distances shorter than 7 are shown in bold).



(b) Distance graph for $\theta = 7$.

(c) Clique graph.

Figure 10.6 The distance graph (b) for $\theta = 7$ is not quite a clique graph. However, it can be transformed into a clique graph (c) by removing edges (g_1, g_{10}) and (g_1, g_9) .

After transforming the distance graph into the clique graph, our data is partitioned into three clusters

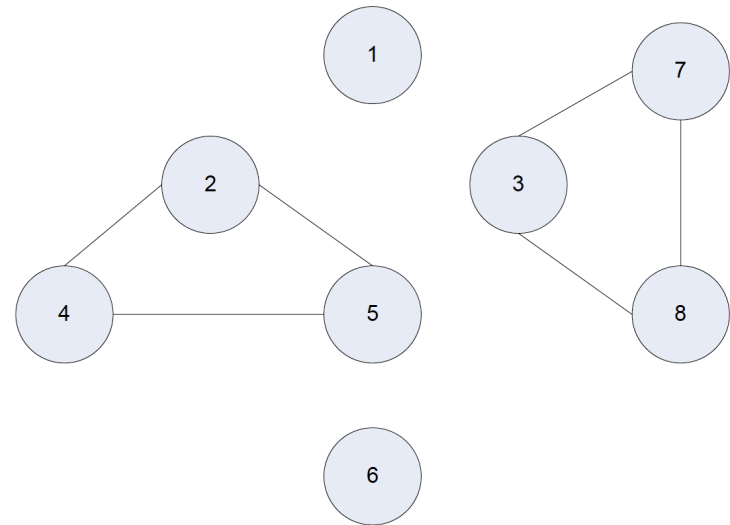
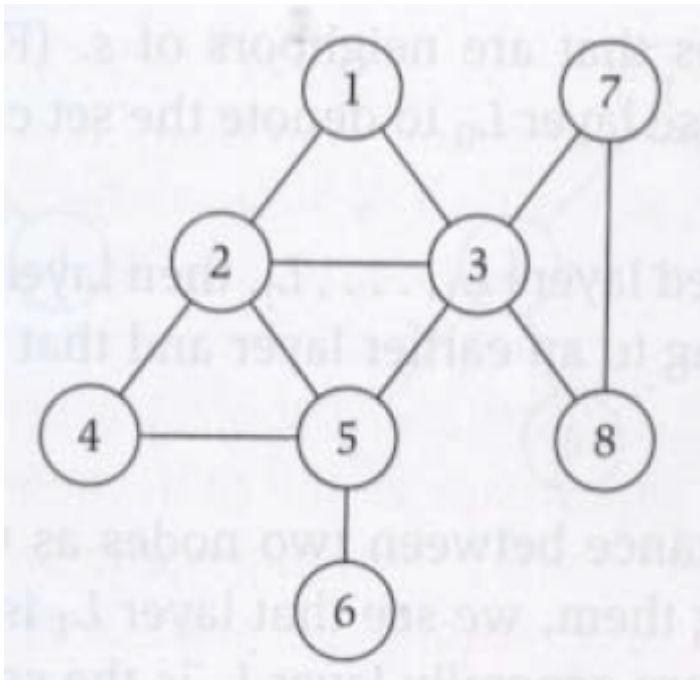
Corrupted Cliques – Example

$$\theta = 6.0$$

	g_2	g_3	g_4	g_5	g_6	g_7	g_8
g_1	5.2	5.1	7.4	7.7	6.3	12.1	11.2
g_2		4.2	3.5	4.6	8.3	6.3	7.2
g_3			7.3	5.2	7.3	3.4	4.9
g_4				5.4	6.5	7.2	10.2
g_5					3.7	7.5	13.2
g_6						14.2	12.1
g_7							4.9

Corrupted Cliques – Example

- Removing 5 edges



Heuristics for Corrupted Clique Problem

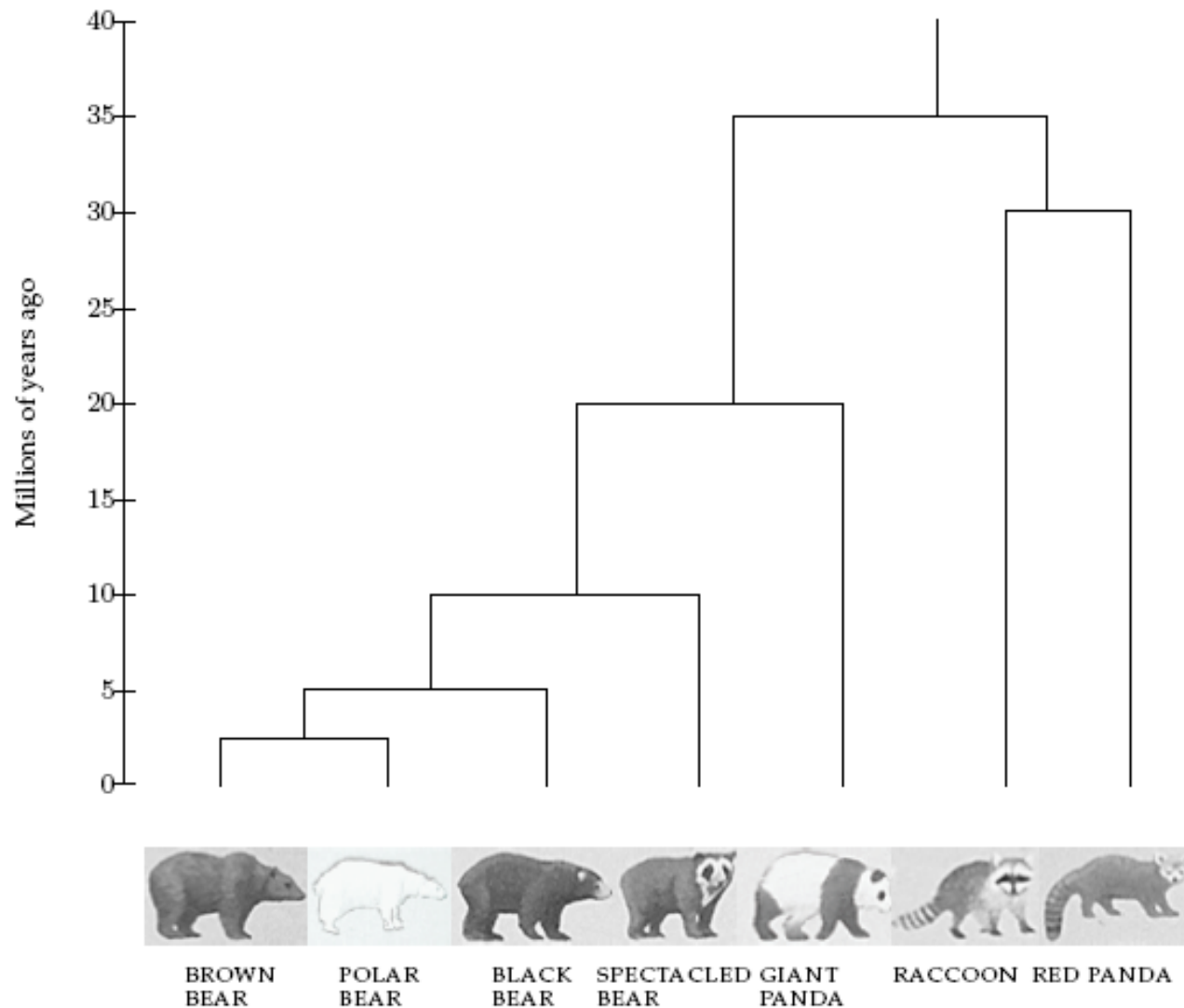
- Corrupted Cliques problem is NP-Hard, some heuristics exist to approximately solve it:
 - **PCC** (Parallel Classification with Cores): a rather slow algorithm
 - **CAST** (Cluster Affinity Search Technique): a practical and faster algorithm inspired by PCC

Molecular Evolution

Rooted Trees

- **Rooted trees:**
 - Infer an evolutionary ancestor
 - leaves represent existing species
 - internal vertices represent hypothetical ancestors
 - can be viewed as directed trees from the root to the leaves

Rooted Trees



Unrooted Trees

- **Unrooted trees:**
 - DOES NOT infer an evolutionary ancestor, therefore cannot be viewed as a directed graph
 - Otherwise, they are like rooted trees

Rooted and Unrooted Trees

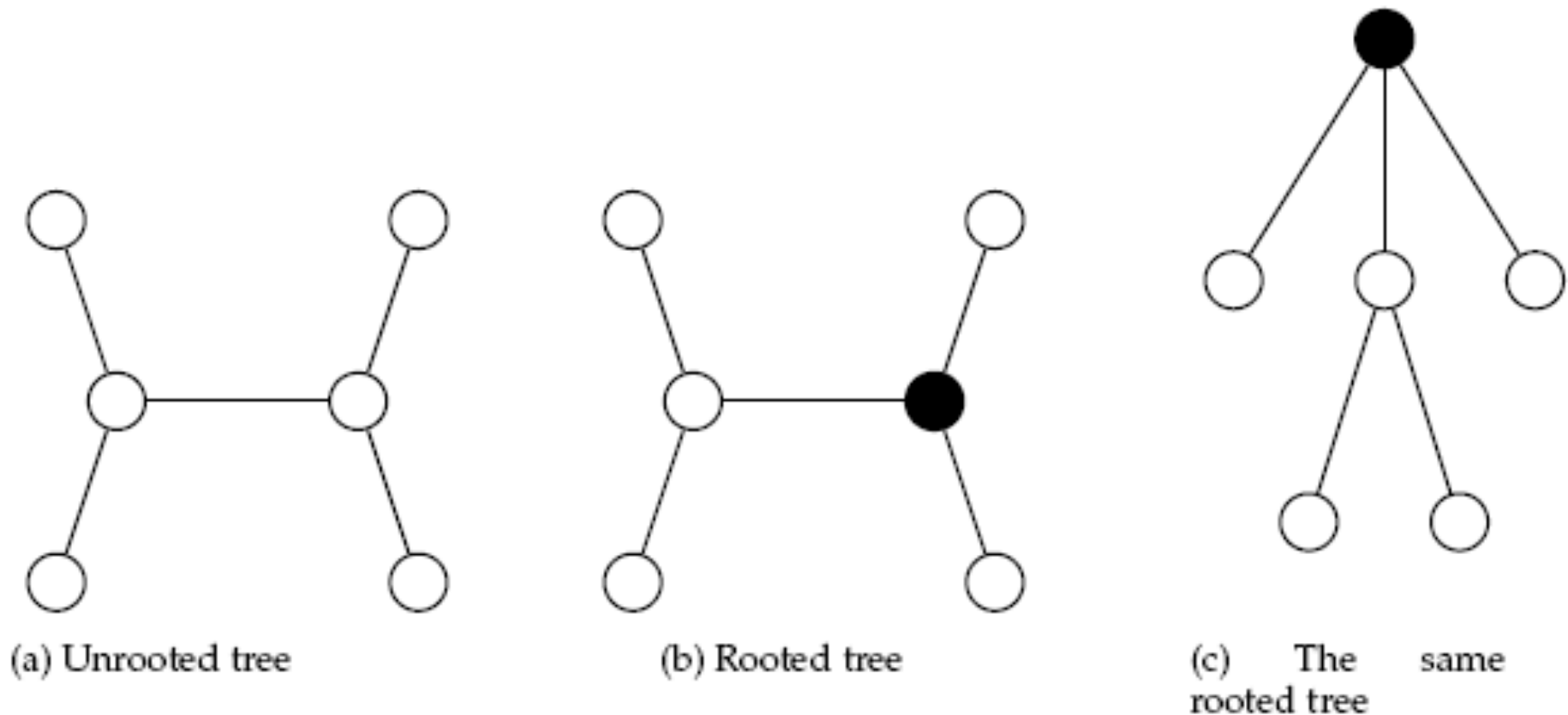
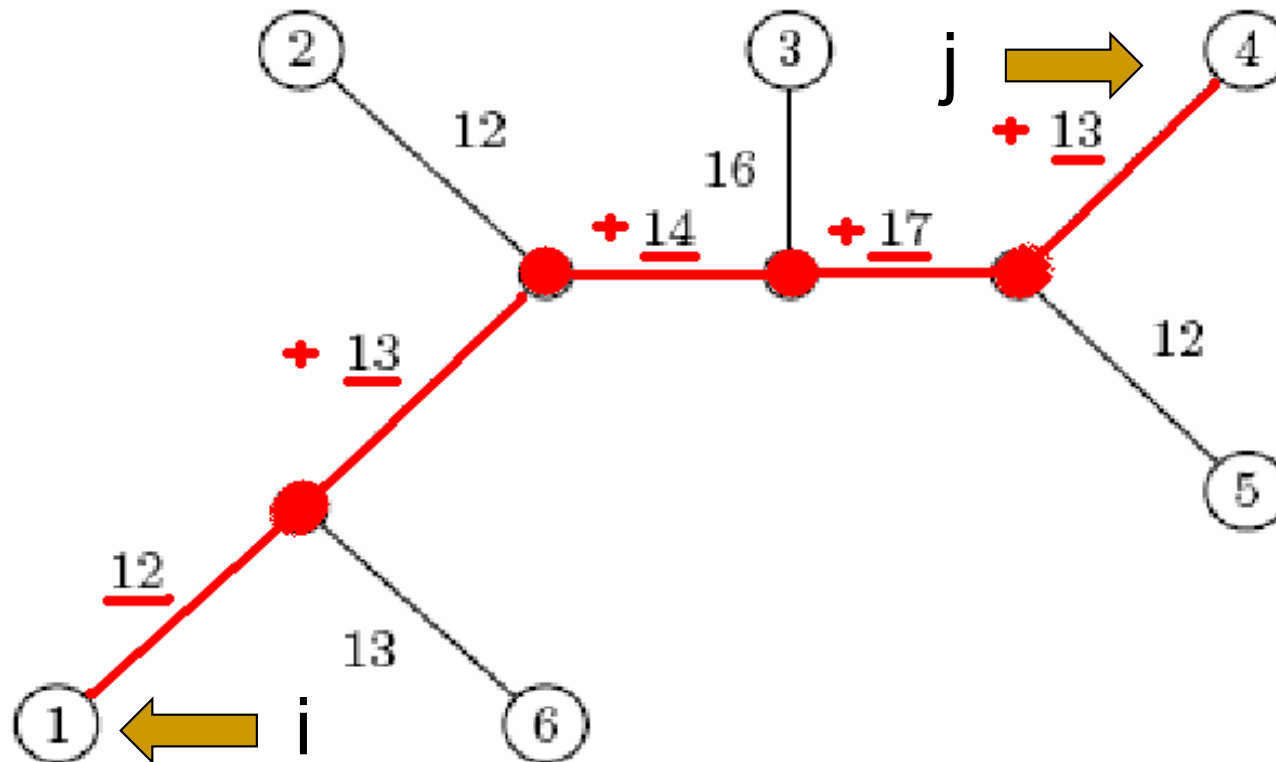


Figure 10.8 The difference between unrooted (a) and rooted (b) trees. These both describe the same tree, but the unrooted tree makes no assumption about the origin of species. Rooted trees are often represented with the root vertex on the top (c), emphasizing that the root corresponds to the ancestral species.

Distances in Trees

- Edges may have weights, which reflect:
 - Number of mutations on evolutionary path from one species to another
 - Or, time estimate for evolution of one species into another
- In a tree T with n leaves, we often compute the length of a path between leaves i and j , $d_{ij}(T)$
 - d_{ij} refers the the distance between i and j is the sum of the weight of the edges between i and j

Distance in Trees (cont' d)



For $i = 1, j = 4$, d_{ij} is:

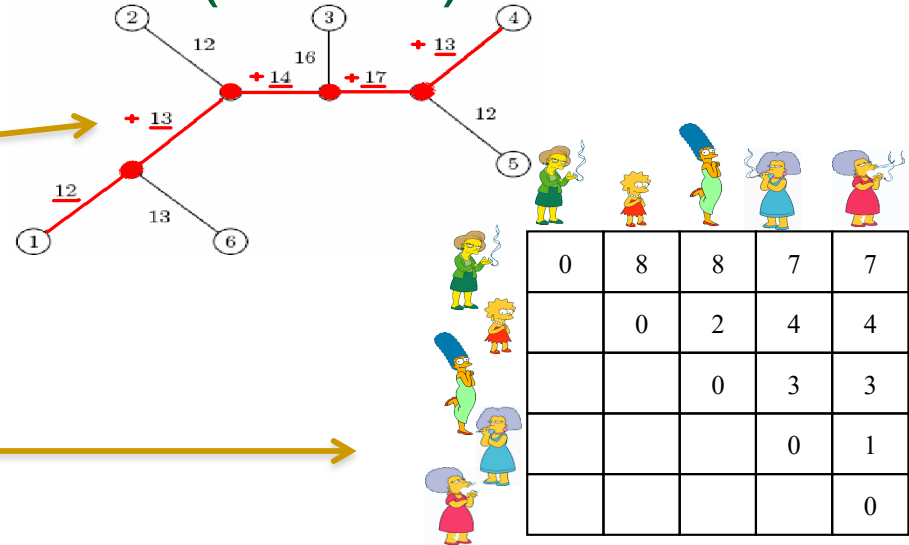
$$d(1,4) = 12 + 13 + 14 + 17 + 13 = 69$$

Fitting Distance Matrix

- Given n species, we can compute the $n \times n$ distance matrix D_{ij}
- There's a corresponding tree, but we don't know the tree.
- With such a matrix we need a method to design a tree that fits this distance matrix D_{ij}

Fitting Distance Matrix (cont' d)

Lengths of paths in a tree T



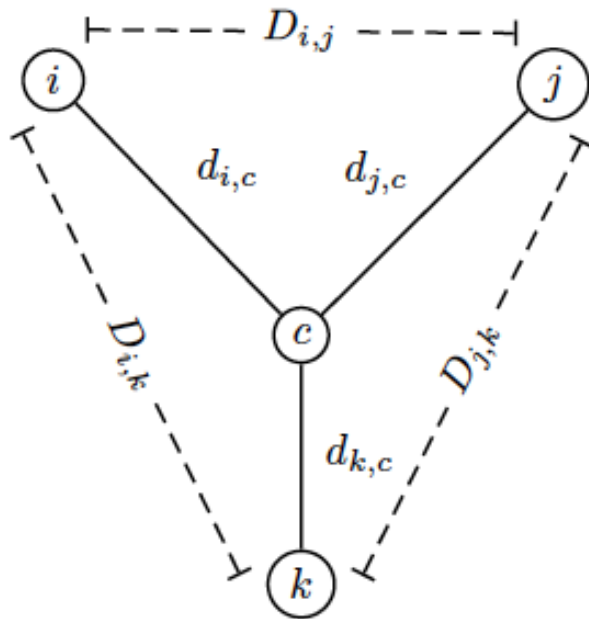
- Fitting means $D_{ij} = d_{ij}(T)$

Edit distance between species

- D_{ij} may be defined as the edit distance between a gene in species i and species j , where the gene of interest is sequenced for all n species

Reconstructing a 3 Leaved Tree

- Tree reconstruction for any 3x3 matrix is straightforward
- We have 3 leaves i, j, k and a center vertex c



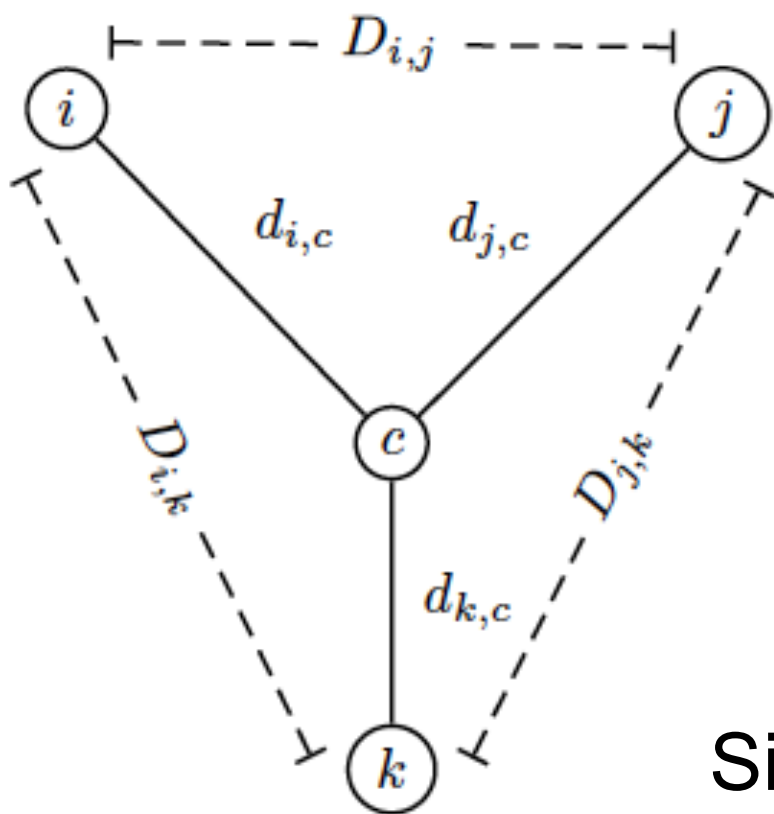
Observe:

$$d_{ic} + d_{jc} = D_{ij}$$

$$d_{ic} + d_{kc} = D_{ik}$$

$$d_{jc} + d_{kc} = D_{jk}$$

Reconstructing a 3 Leaved Tree (cont' d)



$$d_{ic} + d_{jc} = D_{ij}$$

$$+ \underline{d_{ic} + d_{kc} = D_{ik}}$$

$$2d_{ic} + \underbrace{d_{jc} + d_{kc}}_{D_{jk}} = D_{ij} + D_{ik}$$

$$2d_{ic} + D_{jk} = D_{ij} + D_{ik}$$

$$d_{ic} = (D_{ij} + D_{ik} - D_{jk})/2$$

Similarly,

$$d_{jc} = (D_{ij} + D_{jk} - D_{ik})/2$$

$$d_{kc} = (D_{ki} + D_{kj} - D_{ij})/2$$

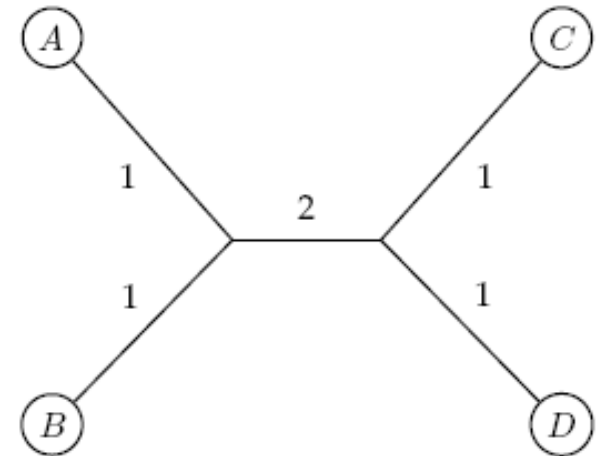
Trees with > 3 Leaves

- An unrooted tree with n leaves has $2n-3$ edges
- This means fitting a given tree to an unknown distance matrix D requires solving a system of all “ n choose 2” pairs of equations with $2n-3$ variables
- This is not always possible to solve for $n > 3$

Additive Distance Matrices

ADDITIVE if there
exists a tree T with
 $d_{ij}(T) = D_{ij}$ 

	A	B	C	D
A	0	2	4	4
B	2	0	4	4
C	4	4	0	2
D	4	4	2	0



NONADDITIVE
otherwise 

	A	B	C	D
A	0	2	2	2
B	2	0	3	2
C	2	3	0	2
D	2	2	2	0

?

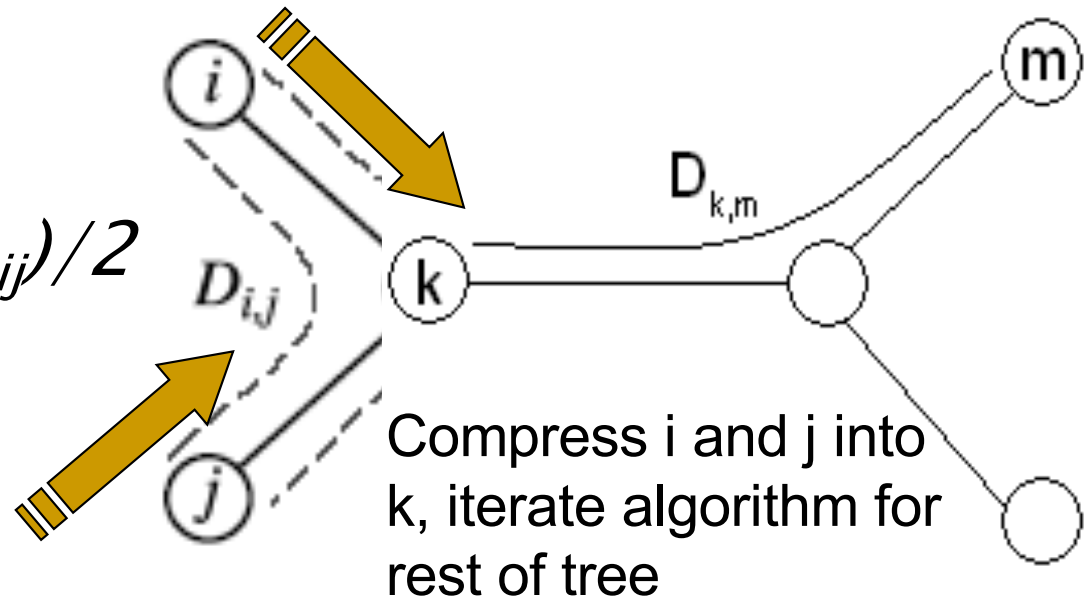
Distance Based Phylogeny Problem

- Goal: Reconstruct an evolutionary tree from a distance matrix
- Input: $n \times n$ distance matrix D_{ij}
- Output: weighted unrooted tree T with n leaves fitting D
- If D is additive, this problem has a solution and there is a simple algorithm to solve it

Neighboring Leaves

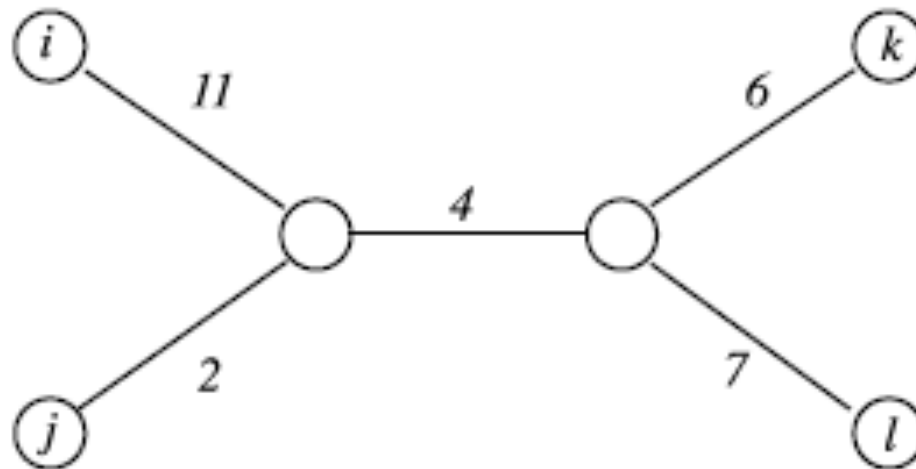
- Find neighboring leaves i and j with parent k
- Remove the rows and columns of i and j
- Add a new row and column corresponding to k , where the distance from k to any other leaf m can be computed by:

$$D_{km} = (D_{im} + D_{jm} - D_{ij})/2$$



Finding Neighboring Leaves

- Closest leaves aren't necessarily neighbors
- i and j are neighbors, but $(d_{ij} = 13) > (d_{jk} = 12)$



- Finding a pair of neighboring leaves is a nontrivial problem!

Degenerate Triples

- A degenerate triple is a set of three distinct elements $1 \leq i, j, k \leq n$ where $D_{ij} + D_{jk} = D_{ik}$
- Found when j is collapsed to 0
- Shows j lies on the evolutionary path from i to k

Finding Degenerate Triples

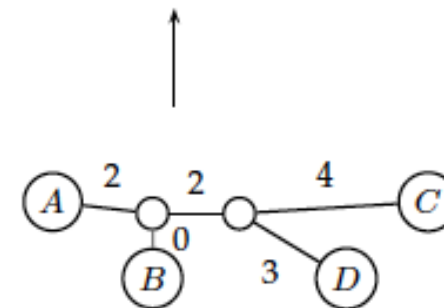
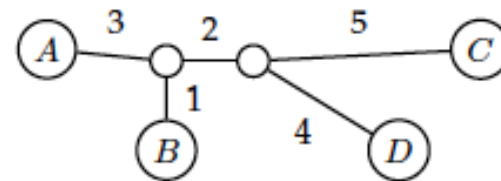
- Shorten all “hanging” edges (edges that connect leaves) until a degenerate triple is found

	A	B	C	D
A	0	4	10	9
B	4	0	8	7
C	10	8	0	9
D	9	7	9	0

$\delta = 1$

	A	B	C	D
A	0	2	8	7
B	2	0	6	5
C	8	6	0	7
D	7	5	7	0

$i \leftarrow A$
 $j \leftarrow B$
 $k \leftarrow C$



Finding Degenerate Triples (cont' d)

- If there is no degenerate triple, all hanging edges are reduced by the same amount δ , so that all pairwise distances in the matrix are reduced by 2δ
- Eventually this process collapses one of the leaves down to 0 (when $\delta = \text{length of shortest hanging edge}$), forming a degenerate triple and reducing our matrix

Reconstructing Tree with Degenerate Triples

- The attachment point for j can be recovered in the reverse transformations by saving D_{ij} for each collapsed edge
- These ideas motivate the following recursive algorithm, AdditivePhylogeny...

Reconstructing Tree with Degenerate Triples (cont' d)

	A	B	C	D
A	0	4	10	9
B	4	0	8	7
C	10	8	0	9
D	9	7	9	0

$\delta = 1$

	A	B	C	D
A	0	2	8	7
B	2	0	6	5
C	8	6	0	7
D	7	5	7	0

$i \leftarrow A$
 $j \leftarrow B$
 $k \leftarrow C$

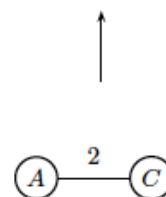
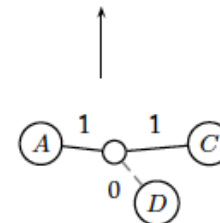
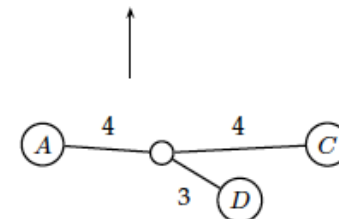
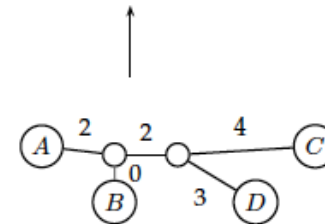
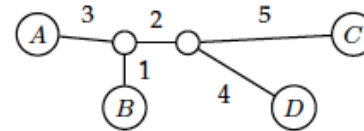
	A	C	D
A	0	8	7
C	8	0	7
D	7	7	0

$\delta = 3$

	A	C	D
A	0	2	1
C	2	0	1
D	1	1	0

$i \leftarrow A$
 $j \leftarrow D$
 $k \leftarrow C$

	A	C
A	0	2
C	2	0



AdditivePhylogeny Algorithm

AdditivePhylogeny(D)

1. If D is a 2×2 matrix
2. T = tree of a single edge of length $D_{1,2}$
3. Return T
4. If D is non-degenerate
5. δ = trimming parameter of matrix D
6. For all $1 \leq i \neq j \leq n$
7. $D_{ij} = D_{ij} - 2 \delta$
8. Else
9. $\delta = 0$

AdditivePhylogeny (cont' d)

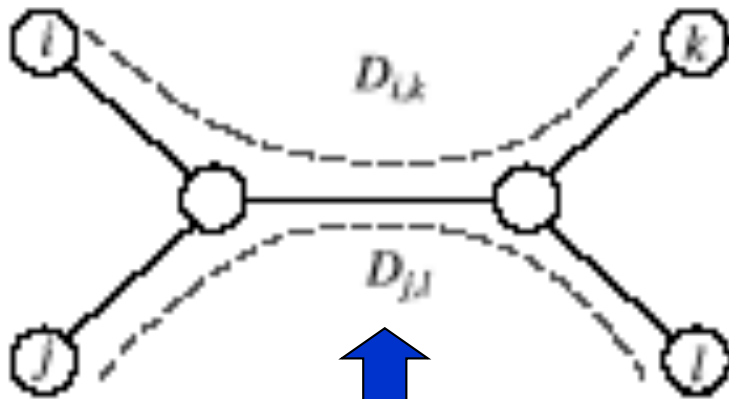
10. Find a triple i, j, k in D such that $D_{ij} + D_{jk} = D_{ik}$
11. $x = D_{ij}$
12. Remove j^{th} row and j^{th} column from D
13. $T = \text{AdditivePhylogeny}(D)$
14. Add a new vertex v to T at distance x from i to k
15. Add j back to T by creating an edge (v, j) of length 0
16. For every leaf l in T
17. If distance from l to v in the tree $\neq D_{l,j}$
18. Output “matrix is not additive”
19. return
20. Extend “hanging” edge leading to all leaves by length δ
21. Return T

The Four Point Condition

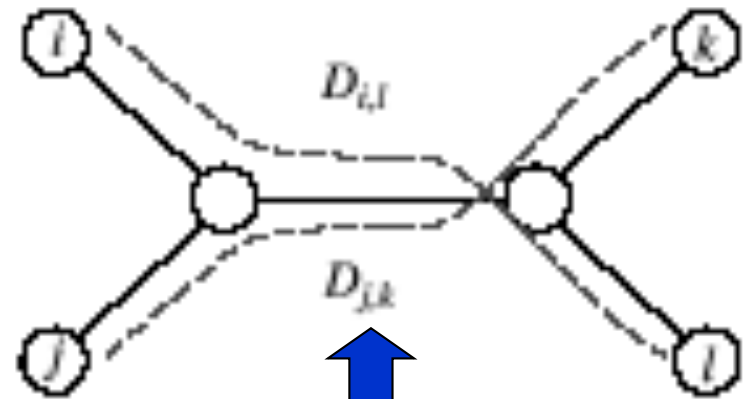
- AdditivePhylogeny provides an **inefficient** way to check if matrix D is additive
- **A more efficient additivity check is the “four-point condition”**
- Let $1 \leq i, j, k, l \leq n$ be four distinct leaves in a tree...

The Four Point Condition (cont' d)

Compute: 1. $D_{ij} + D_{kl}$, 2. $D_{ik} + D_{jl}$, 3. $D_{il} + D_{jk}$

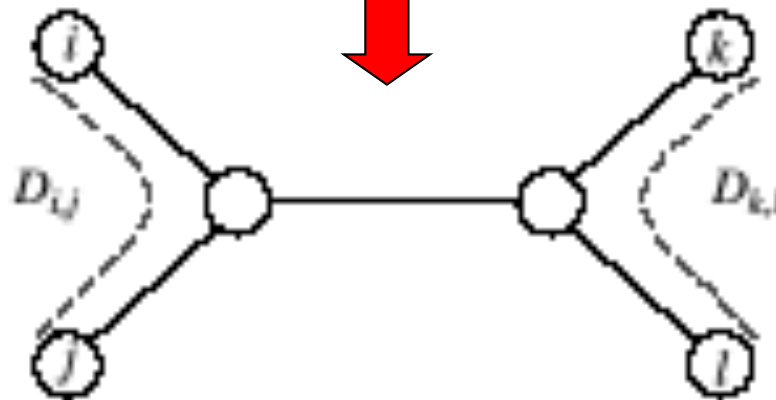


2



3

1



2 and 3
represent the
same number:
the length of all
edges + the
middle edge (its
counted twice)

1 represents a
smaller
number: the
length of all
edges – the
middle edge

The Four Point Condition (cont' d)

- The four point condition is satisfied if 2 of these sums are the same number, with the third sum smaller than these first two
- Theorem : An $n \times n$ matrix D is additive if and only if the four point condition holds for every 4 distinct elements $1 \leq i, j, k, l \leq n$

Introducing Squared Error

- If the distance matrix D of interest is NOT additive, then we naturally look for a tree that approximates D the best
- Squared Error : $\sum_{i,j} (d_{ij}(T) - D_{ij})^2$

Least Squares Distance Phylogeny Problem

- Squared Error is a measure of the quality of the approximation tree: the lower the better, so naturally we want to minimize it
- Finding the best approximation tree T for a non-additive matrix D is NP-hard, and is called the **Least Squares Distance Phylogeny Problem**

Construct an additive tree for:

	A	B	C	D	E
A	0	6	13	15	11
B		0	11	13	9
C			0	12	8
D				0	10
E					0

UPGMA

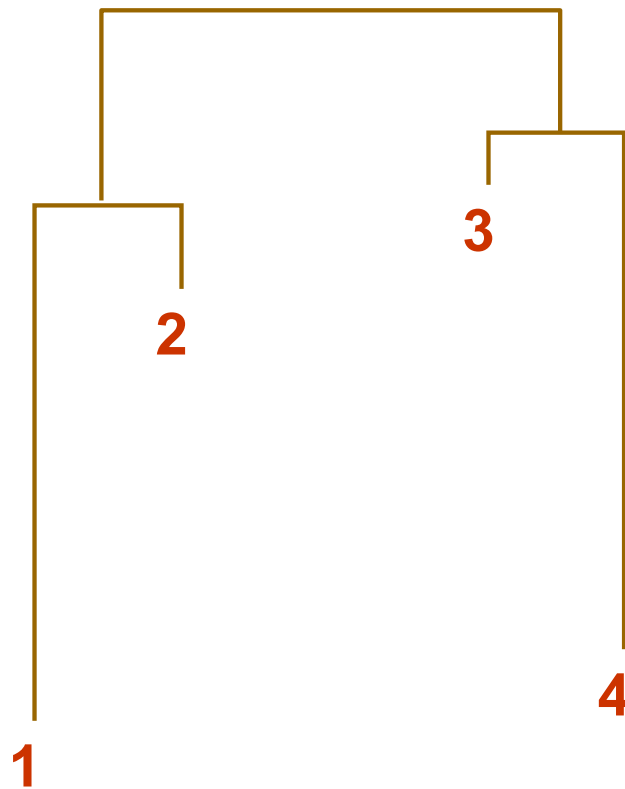
- UPGMA is a clustering algorithm that:
 - computes the distance between clusters using average pairwise distance
 - assigns a height to every vertex in the tree, effectively assuming the presence of a molecular clock and dating every vertex

UPGMA' s Weakness

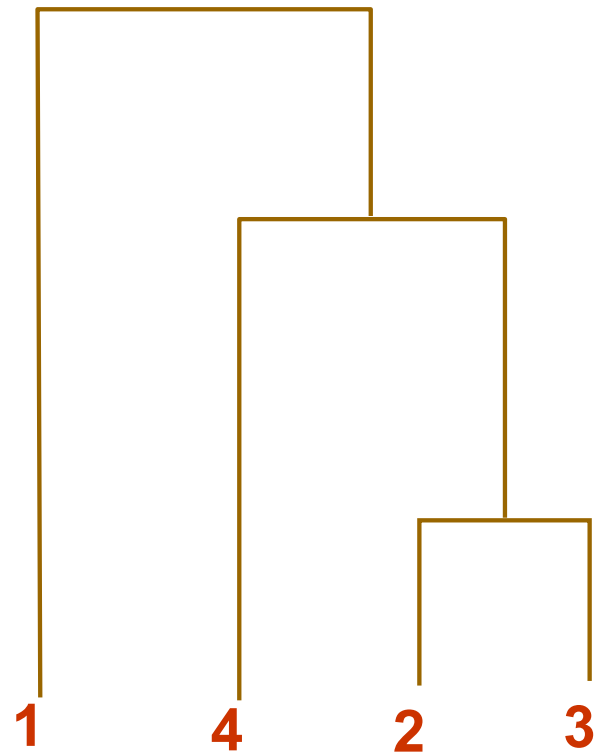
- The algorithm produces an ultrametric tree : the distance from the root to any leaf is the same
 - UPGMA assumes a constant molecular clock: all species represented by the leaves in the tree are assumed to accumulate mutations (and thus evolve) at the same rate. This is one of the major pitfalls of UPGMA.

UPGMA's Weakness: Example

Correct tree



UPGMA



Clustering in UPGMA

Given two disjoint clusters C_i, C_j of sequences,

$$d_{ij} = \frac{1}{|C_i| \times |C_j|} \sum_{\{p \in C_i, q \in C_j\}} d_{pq}$$

Note that if $C_k = C_i \cup C_j$, then distance to another cluster C_l is:

$$d_{kl} = \frac{d_{il} |C_i| + d_{jl} |C_j|}{|C_i| + |C_j|}$$

UPGMA Algorithm

Initialization:

Assign each x_i into its own cluster C_i

Define one leaf per sequence, height 0

Iteration:

Find two clusters C_i, C_j s.t. d_{ij} is min

Let $C_k = C_i \cup C_j$

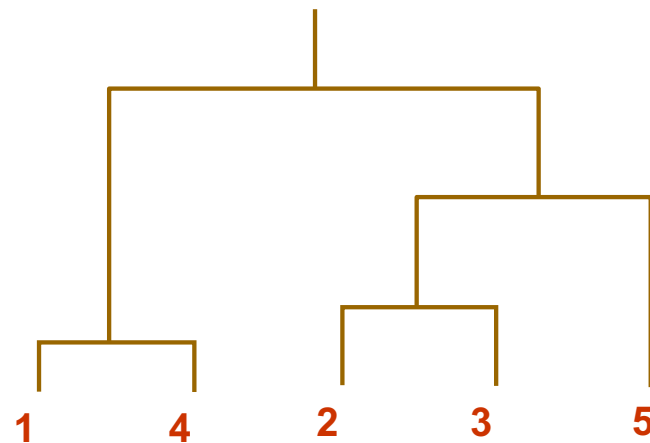
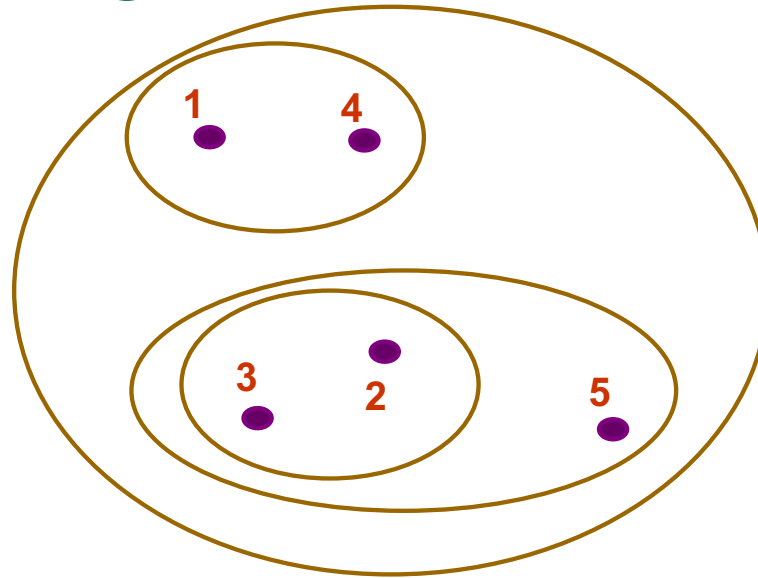
Define node connecting C_i, C_j , & place it at height $d_{ij}/2$

Delete C_i, C_j

Termination:

When two clusters i, j remain, place root at height $d_{ij}/2$

UPGMA Algorithm (cont' d)



Neighbor Joining Algorithm

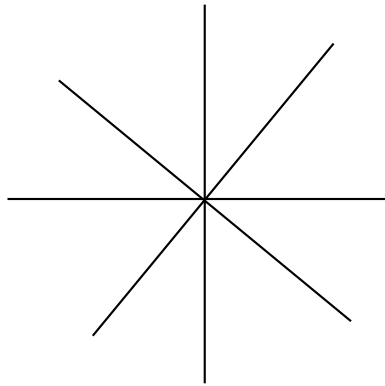
- In 1987 Naruya Saitou and Masatoshi Nei developed a neighbor joining algorithm for phylogenetic tree reconstruction
- **Finds clusters of leaves that are simultaneously close to each other but far from others**

Neighbor Joining Algorithm (cont' d)

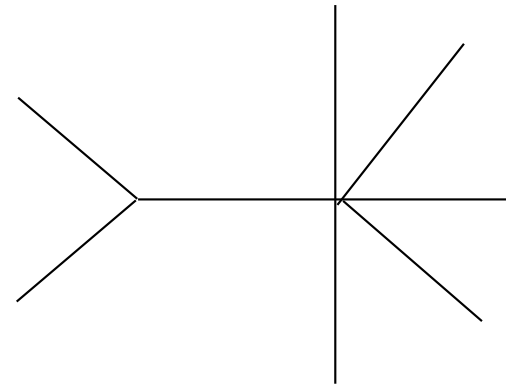
- Advantages: works well for additive and other non-additive matrices.
- Neighbor Joining algorithm implicitly finds a pair of neighboring leaves

Neighbor Joining Algorithm (cont' d)

1. Start with a star tree – no topology



2. Choose a pair of objects that are simultaneously close to each other and far from others.



3. This pair is reconsidered as single object and distance matrix is recalculated.

4. Go to step 2

5. Iterate until complete.

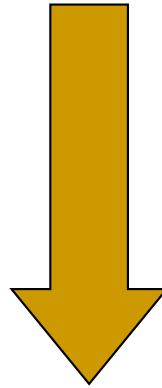
Neighbor Joining Algorithm (cont' d)

NEIGHBORJOINING(D, n)

- 1 Form n clusters, each with a single element
- 2 Construct a graph T by assigning an isolated vertex to each cluster
- 3 while there is more than one cluster
- 4 Find clusters C_1 and C_2 minimizing $D(C_1, C_2) - u(C_1) - u(C_2)$
- 5 Merge C_1 and C_2 into a new cluster C with $|C_1| + |C_2|$ elements
- 6 Compute $D(C, C^*) = \frac{D(C_1, C) + D(C_2, C)}{2}$ to every other cluster C^*
- 7 Add a new vertex C to T and connect it to vertices C_1 and C_2
- 8 Assign length $\frac{1}{2}D(C_1, C_2) + \frac{1}{2}(u(C_1) - u(C_2))$ to the edge (C_1, C)
- 9 Assign length $\frac{1}{2}D(C_1, C_2) + \frac{1}{2}(u(C_2) - u(C_1))$ to the edge (C_2, C)
- 10 Remove rows and columns of D corresponding to C_1 and C_2
- 11 Add a row and column to D for the new cluster C
- 12 return T

Character-Based Tree Reconstruction

Sequence a gene of length m nucleotides in n species to generate an $n \times m$ alignment matrix



Transform
into...

$n \times n$ distance matrix ...
analyze data by a distance-
based tree reconstruction

Character-Based Tree Reconstruction

(cont' d)

- **Better technique:**
 - Use character-based reconstruction algorithms which use the $n \times m$ alignment matrix ($n = \# \text{ species}$, $m = \# \text{ characters}$) directly instead of using distance-based reconstruction algorithms
- **GOAL:** determine what character strings at internal nodes would best explain the character strings for the n observed species

Character-Based Tree Reconstruction

(cont' d)

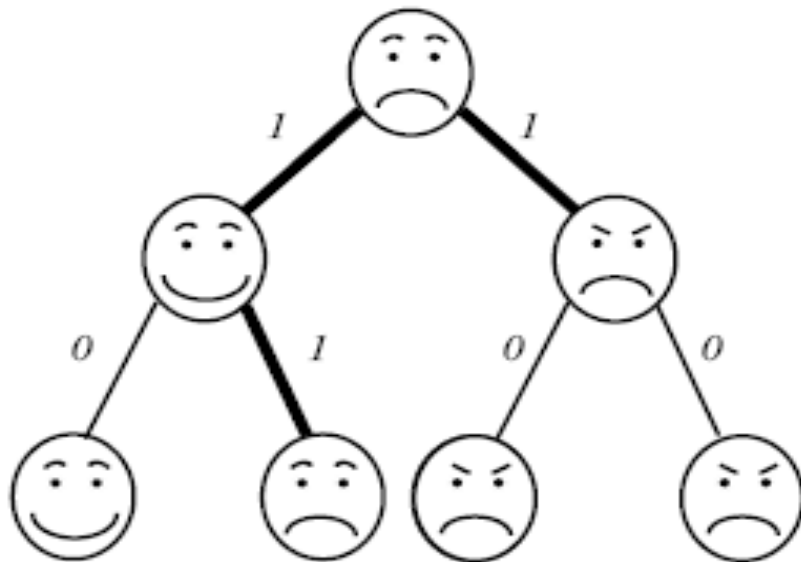
- Given a tree T with every vertex labeled by an m -long string of characters, one can set the length of an edge in the tree to the Hamming distance, we may define the parsimony score of the tree as the sum of the lengths (weights) of the edges

Character-Based Tree Reconstruction

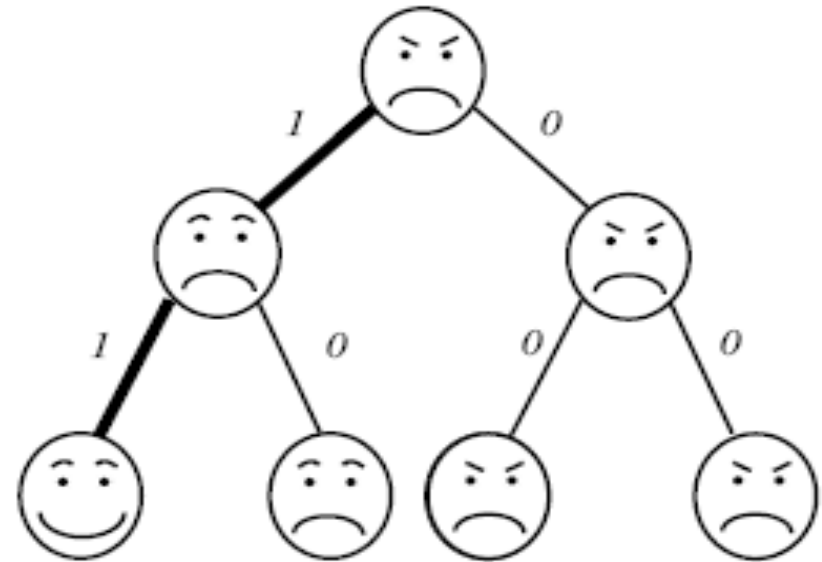
(cont' d)

- The parsimony score can be seen as the total number of mutations required to explain all of the observed characters sequences.
- The parsimony approach attempts to minimize this score, and follows the philosophy of Occam's razor: find the simplest explanation of the data.

Character-Based Tree Reconstruction (cont' d)



(a) *Parsimony Score=3*



(b) *Parsimony Score=2*

Figure 10.16 If we label a tree's leaves with characters (in this case, eyebrows and mouth, each with two states), and choose labels for each internal vertex, we implicitly create a *parsimony* score for the tree. By changing the labels in (a) we are able to create a tree with a better parsimony score in (b).

Small Parsimony Problem

- Input: Tree T with each leaf labeled by an m -character string.
- Output: Labeling of internal vertices of the tree T minimizing the parsimony score.

The Large Parsimony problem assumes neither the tree structure nor the labels are known.

Weighted Small Parsimony Problem

- A more general version of Parsimony Problem
- Input includes a $k * k$ scoring matrix (all entries is 1 if un weighted)
- David Sankoff' s dynamic programming algorithm

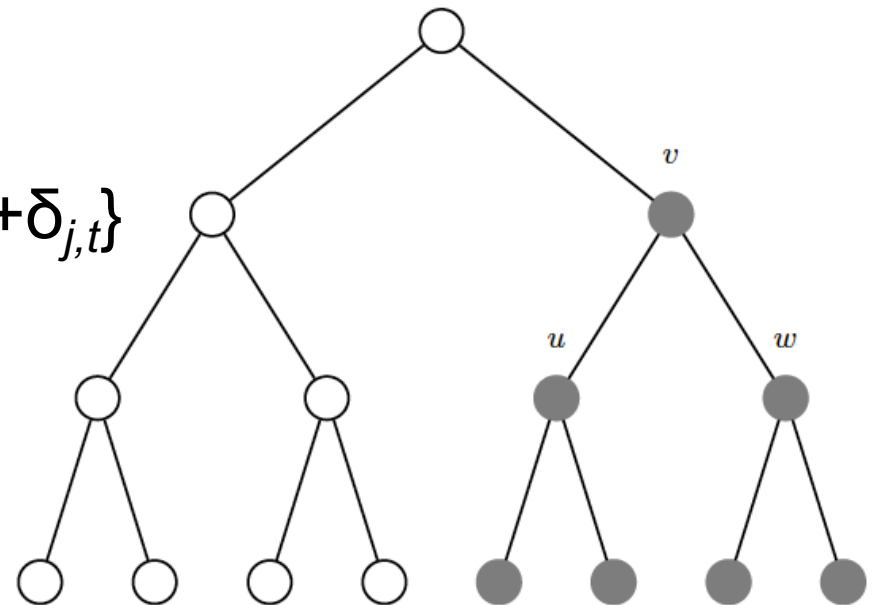
Sankoff' s Algorithm

- Build from bottom leaf to root
- Then move down and assign each node with its character.
- The character at each leave and internal vertex isn' t determined until the root is determined.
- Need to keep tract of every character.

Sankoff's Algorithm (cont'd)

- Dynamic programming
- Let $s_t(v)$ be the minimum parsimony score of the subtree of v under the assumption that vertex v has character t .
- $s_t(v)$ can be computed by analyzing k scores $s_i(u)$ and k scores $s_i(w)$ for $1 \leq i \leq k$.

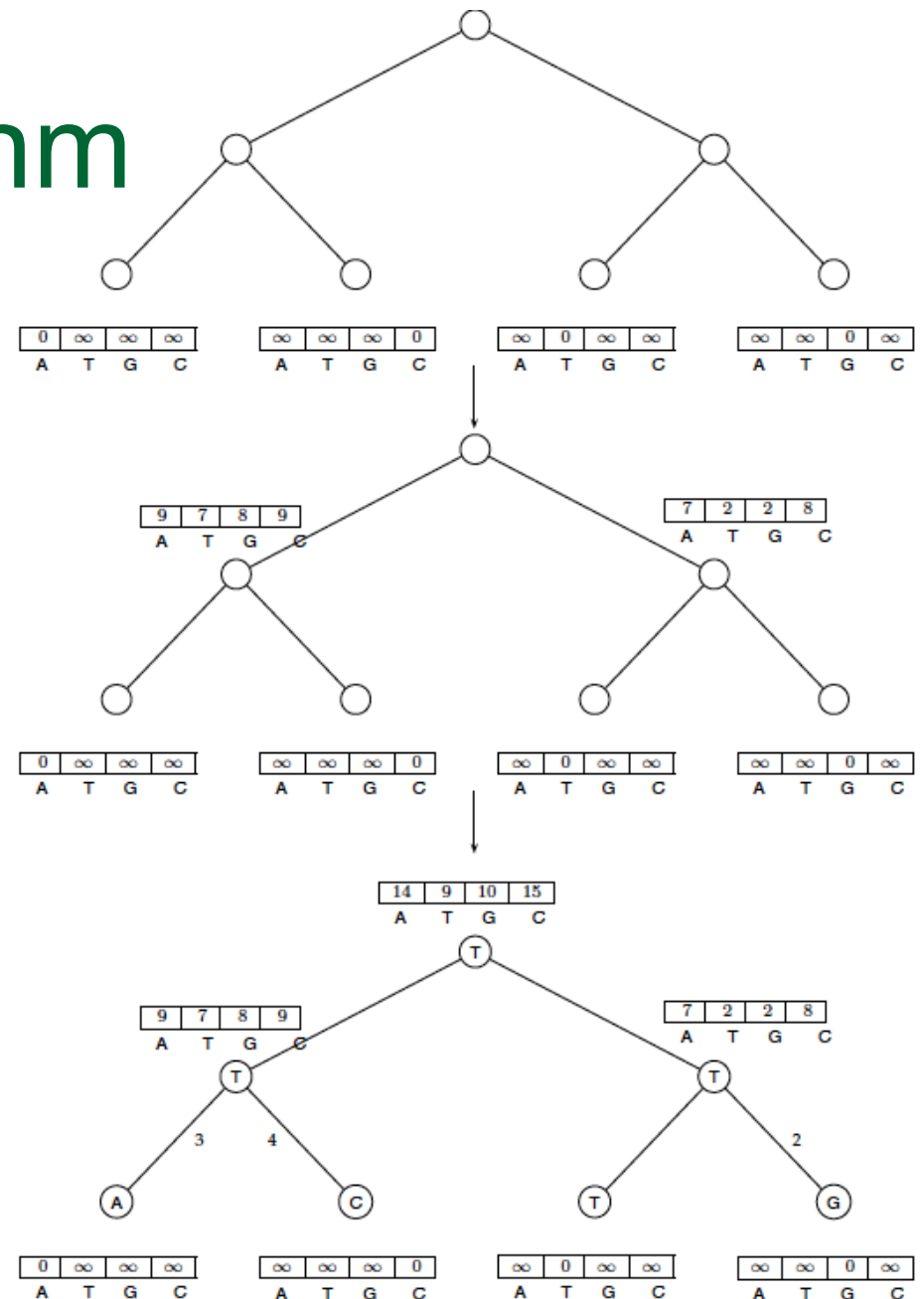
$$s_t(v) = \min_i \{s_i(u) + \delta_{i,t}\} + \min_j \{s_j(w) + \delta_{j,t}\}$$



Sankoff's Algorithm

- Check children's every character and determine the minimum between them
- An example

δ	A	T	G	C
A	0	3	4	9
T	3	0	2	4
G	4	2	0	4
C	9	4	4	0



Fitch's Algorithm

- Solves unweighted small Parsimony problem
- Dynamic programming in essence
- Assigns a set of letter to every vertex in the tree.
- If the two children's sets of character overlap, it's the common set of them
- If not, it's the combined set of them.

Fitch's Algorithm (cont' d)

An example:

